



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΣΥΣΤΗΜΑΤΩΝ ΒΑΣΕΩΝ ΓΝΩΣΕΩΝ ΚΑΙ ΔΕΔΟΜΕΝΩΝ
Ακ. έτος 2025-2026, 6ο εξάμηνο, ΣΗΜΜΥ

Εξαμηνιαία Εργασία: «Σχεδιασμός και Υλοποίηση Πληροφοριακού Συστήματος Διαχείρισης του Γενικού Νοσοκομείου "Υγειόπολης"»

ΜΕΛΗ

ΒΑΛΛΑΣ ΔΗΜΗΤΡΙΟΣ 03123190

ΠΟΥΛΙΟΥ ΕΛΕΝΗ 031232109

ΚΟΚΚΙΝΟΥ ΣΤΥΛΙΑΝΗ ge22034

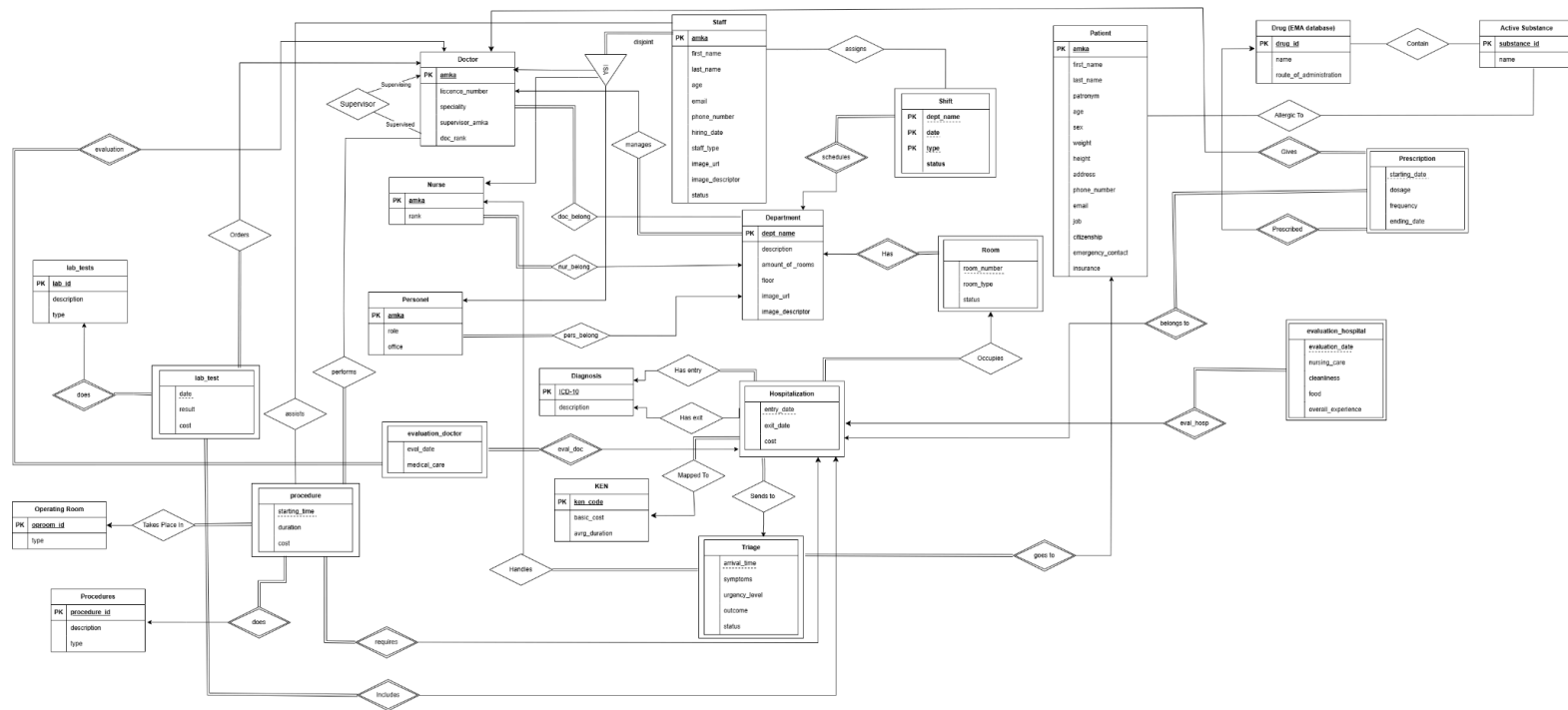
ΕΙΣΑΓΩΓΗ

Η παρούσα εργαστηριακή αναφορά αφορά τη σχεδίαση, μοντελοποίηση και υλοποίηση μιας σχεσιακής βάσης δεδομένων για τη διαχείριση των λειτουργιών του Γενικού Νοσοκομείου «Υγειόπολις». Αντικείμενο του συστήματος είναι η αποθήκευση και η αποδοτική διαχείριση των πληροφοριών που αφορούν το ιατρονοσηλευτικό και διοικητικό προσωπικό, τις κλινικές και τους θαλάμους, τους ασθενείς, τη διαδικασία διαλογής (triage) στα επείγοντα, καθώς και τις ιατρικές εξετάσεις και νοσηλείες.

Η διαδικασία ανάπτυξης ακολούθησε όλα τα στάδια σχεδιασμού: από την εννοιολογική μοντελοποίηση με τη χρήση του Διαγράμματος Οντοτήτων-Συσχετίσεων (E/R), στον λογικό σχεδιασμό (Σχεσιακό Μοντέλο) και τελικά στη φυσική υλοποίηση με τη χρήση της SQL (MySQL/MariaDB), εμπλουτισμένης με περιορισμούς ακεραιότητας και όψεις (views).

Εννοιολογικός Σχεδιασμός (Διάγραμμα E/R)

Παρακάτω φαίνεται το ER διάγραμμα της βάσης δεδομένων μας το οποίο περιλαμβάνει οντότητες όπως :



Το παραπάνω διάγραμμα εμφανίζονται όλες οι βασικές οντότητες με τα χαρακτηριστικά τους, όπως ορίζονται στην εκφώνηση, καθώς και τις μεταξύ τους σχέσεις. Τα entities που συμβολίζονται με διπλό ορθογώνιο είναι *weak entities*. Αυτές οι οντότητες δεν διαθέτουν τα κατάλληλα attributes για να ορίσουν το primary key τους και χρειάζεται να εξαρτώνται από μία άλλη ισχυρή οντότητα μέσω μίας σχέσης που ονομάζεται *identifying relationship*.

Παρακάτω παρουσιάζονται οι σχέσεις μεταξύ των οντοτήτων:

- **STAFF-DOCTOR/NURSE/PERSONNEL (σχέση ISA)**

Το προσωπικό του νοσοκομείου (**staff**) διακρίνεται σε τρεις υποκατηγορίες: ιατρός (**doctor**), νοσηλεύτης (**nurse**), διοικητικό προσωπικό (**personnel**). Η σχέση μεταξύ τους αποτυπώνεται με εξειδίκευση τύπου ISA, με supertype το staff και subtypes τα doctor, nurse, personnel. Η εξειδίκευση είναι total (κάθε μέλος του προσωπικού πρέπει να ανήκει υποχρεωτικά σε μία υποκατηγορία) και disjoint (κανένα μέλος του προσωπικού δεν μπορεί να ανήκει ταυτόχρονα σε πάνω από μία υποκατηγορίες).

- **DOCTOR-DOCTOR (αυτοαναφορική σχέση-SUPERVISOR)**

Κάθε ιατρός μπορεί να έχει έναν επόπτη (**supervisor**), που είναι επίσης ιατρός, και ένας ιατρός μπορεί να επιβλέπει πολλούς άλλους ιατρούς. Συνεπώς, η σχέση είναι '**N:1**' στο **doctor**, με ρόλους "*Supervising*" και "*Supervised*". Και από τις δύο μεριές έχουμε μερική συμμετοχή, καθώς οι Διευθυντές δεν έχουν επόπτη ενώ δεν έχουν υποχρεωτικά όλοι οι ιατροί supervisor.

- **DOCTOR-DEPARTMENT (κανονική σχέση-MANAGES)**

Κάθε τμήμα έχει ακριβώς έναν διευθυντή, ο οποίος είναι ιατρός (**doctor**), και ένας ιατρός μπορεί να διευθύνει το πολύ ένα τμήμα (**department**). Συνεπώς, η σχέση αυτή είναι '**1:1**' από **DOCTOR** προς **DEPARTMENT** με ολική συμμετοχή από την πλευρά του department και μερική συμμετοχή από την πλευρά του staff.

- **DOCTOR-DEPARTMENT (κανονική σχέση-DOC_BELONG)**

Κάθε ιατρός ανήκει σε ένα ή περισσότερα τμήματα (**department**), και κάθε τμήμα περιλαμβάνει πολλούς ιατρούς (**doctor**). Συνεπώς, η σχέση αυτή είναι '**N:M**' από **DOCTOR** προς **DEPARTMENT** με ολική συμμετοχή από την πλευρά του doctor.

- **NURSE-DEPARTMENT (κανονική σχέση-NUR_BELONG)**

Κάθε νοσηλευτής ανήκει σε ακριβώς ένα τμήμα (**department**), ενώ κάθε τμήμα περιλαμβάνει πολλούς νοσηλευτές (**nurse**). Συνεπώς, η σχέση αυτή είναι 'N:1' από **NURSE** προς **DEPARTMENT** με ολική συμμετοχή από την πλευρά του nurse.

- **PERSONNEL-DEPARTMENT (κανονική σχέση-PERS_BELONG)**

Κάθε μέλος του διοικητικού προσωπικού (**personnel**) ανήκει σε ακριβώς ένα τμήμα, ενώ κάθε τμήμα (**department**) περιλαμβάνει πολλά μέλη του διοικητικού προσωπικού. Συνεπώς, η σχέση αυτή είναι 'N:1' από **PERSONNEL** προς **DEPARTMENT** με ολική συμμετοχή από την πλευρά του personnel.

- **DEPARTMENT-ROOM (identifying σχέση-HAS)**

Κάθε τμήμα διαθέτει έναν αριθμό κλινών (**room**), και κάθε κλίνη ανήκει σε ακριβώς ένα τμήμα (**department**). Συνεπώς, η σχέση αυτή είναι '1:N' από **DEPARTMENT** προς **ROOM** με ολική συμμετοχή από την πλευρά του room.

- **STAFF-SHIFT (κανονική σχέση-ASSIGNS)**

Κάθε μέλος προσωπικού μπορεί να συμμετέχει σε πολλές βάρδιες (**shift**), και κάθε βάρδια καλύπτεται από πολλά μέλη προσωπικού (**staff**). Συνεπώς, η σχέση αυτή είναι 'N:M' από **STAFF** προς **SHIFT** με μερική συμμετοχή και από τις δύο πλευρές.

- **DEPARTMENT-SHIFT (identifying σχέση-SCHEDULES)**

Κάθε βάρδια αφορά ένα συγκεκριμένο τμήμα (**department**), ενώ κάθε τμήμα έχει πολλές βάρδιες (**shift**). Συνεπώς, η σχέση αυτή είναι 'N:1' από **SHIFT** προς **DEPARTMENT** με ολική συμμετοχή από την πλευρά του shift.

- **PATIENT-TRIAGE (identifying σχέση-GOES TO)**

Κάθε ασθενής που καταφθάνει στο νοσοκομείο υποβάλλεται σε διαλογή (**triage**), και κάθε ασθενής (**patient**) μπορεί να έχει πολλαπλές διαλογές στο ιστορικό του. Συνεπώς, η σχέση αυτή είναι '1:N' από **PATIENT** προς **TRIAGE** με ολική συμμετοχή από την πλευρά του triage.

- **NURSE-TRIAGE (κανονική σχέση-HANDLES)**

Κάθε διαλογή εκτελείται από έναν συγκεκριμένο νοσηλευτή (**nurse**), ενώ ένας νοσηλευτής μπορεί να εκτελεί πολλές διαλογές (**triage**). Συνεπώς, η σχέση αυτή είναι 'N:1' από **TRIAGE** προς **NURSE** με ολική συμμετοχή από την πλευρά του triage.

- **TRIAGE–HOSPITALIZATION (κανονική σχέση-SENDS TO)**

Μία διαλογή μπορεί (ή όχι) να οδηγήσει σε νοσηλεία (**hospitalization**) και κάθε νοσηλεία αντιστοιχίζεται σε μία μόνο διαλογή (**triage**). Συνεπώς, η σχέση αυτή είναι ‘1:1’ από **TRIAGE** προς **HOSPITALIZATION** με ολική συμμετοχή από την πλευρά του hospitalization.

- **HOSPITALIZATION–ROOM (κανονική σχέση-OCCUPIES)**

Κάθε νοσηλεία αντιστοιχεί υποχρεωτικά σε μία συγκεκριμένη κλίνη (**room**), ενώ μία κλίνη μπορεί να φιλοξενεί πολλές νοσηλείες (**hospitalization**) σε διαφορετικές όμως χρονικές περιόδους. Συνεπώς, η σχέση αυτή είναι ‘N:1’ από **HOSPITALIZATION** προς **ROOM** με ολική συμμετοχή από την πλευρά του hospitalization.

- **HOSPITALIZATION–KEN (κανονική σχέση-MAPPED TO)**

Κάθε νοσηλεία αντιστοιχίζεται υποχρεωτικά σε έναν κωδικό KEN (**KEN**), ενώ ένας κωδικός KEN μπορεί να αντιστοιχίζεται σε πολλές νοσηλείες (**hospitalization**). Συνεπώς, η σχέση αυτή είναι ‘N:1’ από **HOSPITALIZATION** προς **KEN** με ολική συμμετοχή από την πλευρά του hospitalization.

- **HOSPITALIZATION–DIAGNOSIS (κανονική σχέση-HAS ENTRY)**

Κάθε νοσηλεία έχει υποχρεωτικά μία διάγνωση εισαγωγής (κωδικός ICD-10) (**diagnosis**), ενώ ο ίδιος κωδικός διάγνωσης μπορεί να αντιστοιχεί σε πολλές νοσηλείες (**hospitalization**). Συνεπώς, η σχέση αυτή είναι ‘N:1’ από **HOSPITALIZATION** προς **DIAGNOSIS** με ολική συμμετοχή από την πλευρά του hospitalization.

- **HOSPITALIZATION–DIAGNOSIS (κανονική σχέση-HAS EXIT)**

Κάθε νοσηλεία μπορεί να έχει μία διάγνωση εξόδου (κωδικός ICD-10) (**diagnosis**), ενώ ο ίδιος κωδικός διάγνωσης μπορεί να αντιστοιχεί σε πολλές νοσηλείες (**hospitalization**) ως διάγνωση εξόδου. Συνεπώς, η σχέση αυτή είναι ‘N:1’ από **HOSPITALIZATION** προς **DIAGNOSIS** με μερική συμμετοχή από την πλευρά του hospitalization.

- **HOSPITALIZATION–LAB_TEST (identifying σχέση-INCLUDES)**

Κάθε νοσηλεία μπορεί να συνδέεται με πολλές εργαστηριακές εξετάσεις (**lab_test**), και κάθε εξέταση ανήκει υποχρεωτικά σε μία συγκεκριμένη νοσηλεία (**hospitalization**). Συνεπώς, η σχέση αυτή είναι ‘1:N’ από **HOSPITALIZATION** προς **LAB_TEST**, με ολική συμμετοχή από την πλευρά του lab_test.

- **LAB_TESTS–LAB_TEST (identifying σχέση-DOES)**

Κάθε εργαστηριακή εξέταση αντιστοιχεί υποχρεωτικά σε έναν τύπο εξέτασης από τον κατάλογο **LAB_TESTS (lab_tests)**, ενώ ένας τύπος εξέτασης μπορεί να αντιστοιχεί σε πολλές εξετάσεις (**lab_test**). Συνεπώς, η σχέση αυτή είναι **‘1:N’** από **LAB_TESTS** προς **LAB_TEST** με ολική συμμετοχή από την πλευρά του lab_test.

- **DOCTOR–LAB_TEST (κανονική σχέση-ORDERS)**

Κάθε εργαστηριακή εξέταση παραγγέλλεται υποχρεωτική από έναν συγκεκριμένο ιατρό (**doctor**), ενώ ένας ιατρός μπορεί να παραγγείλει πολλές εξετάσεις (**lab_test**). Συνεπώς, η σχέση αυτή είναι **‘N:1’** από **LAB_TEST** προς **DOCTOR**, με ολική συμμετοχή από την πλευρά του lab_test.

- **HOSPITALIZATION–PROCEDURE (identifying σχέση-REQUIRES)**

Κάθε νοσηλεία μπορεί να συνδέεται με πολλές ιατρικές πράξεις (**procedure**), και κάθε πράξη ανήκει υποχρεωτικά σε μία συγκεκριμένη νοσηλεία (**hospitalization**). Συνεπώς, η σχέση αυτή είναι **‘1:N’** από **HOSPITALIZATION** προς **PROCEDURE** με ολική συμμετοχή από την πλευρά του procedure.

- **PROCEDURES–PROCEDURE (identifying σχέση-DOES)**

Κάθε ιατρική πράξη αντιστοιχεί υποχρεωτικά σε έναν τύπο πράξης από τον κατάλογο **PROCEDURES (procedures)**, ενώ ένας τύπος πράξης μπορεί να αντιστοιχεί σε πολλές ιατρικές πράξεις (**procedure**). Συνεπώς, η σχέση αυτή είναι **‘1:N’** από **PROCEDURES** προς **PROCEDURE** με ολική συμμετοχή από την πλευρά του procedure.

- **DOCTOR–PROCEDURE (κανονική σχέση-PERFORMES)**

Κάθε ιατρική πράξη εκτελείται από έναν κύριο χειρουργό (**doctor**), ενώ ένας ιατρός μπορεί να εκτελεί πολλές πράξεις (**procedure**). Συνεπώς, η σχέση αυτή είναι **‘N:1’** από **PROCEDURE** προς **DOCTOR** με ολική συμμετοχή από την πλευρά του procedure.

- **OPERATING_ROOM–PROCEDURE (κανονική σχέση-TAKES PLACE IN)**

Κάθε ιατρική πράξη πραγματοποιείται υποχρεωτικά σε μία συγκεκριμένη αίθουσα επέμβασης (**operating_room**), ενώ ένα χειρουργείο μπορεί να φιλοξενεί πολλές πράξεις (**procedure**) σε διαφορετικές όμως χρονικές στιγμές. Συνεπώς, η σχέση αυτή είναι **‘N:1’** από **PROCEDURE** προς **OPERATING_ROOM** με ολική συμμετοχή από την πλευρά του procedure.

- **STAFF-PROCEDURE (κανονική σχέση-ASSISTS)**

Κάθε ιατρική πράξη μπορεί να εκτελείται με τη βοήθεια βοηθών , οι οποίοι είναι μέλη του προσωπικού (**staff**), και ένα μέλος προσωπικού μπορεί να συμμετέχει ως βοηθός σε πολλές πράξεις (**procedure**). Συνεπώς, η σχέση αυτή είναι '**N:M**' από **STAFF** προς **PROCEDURE** με μερική συμμετοχή και από τις δύο πλευρές.

- **DOCTOR-PRESCRIPTION (identifying σχέση-GIVES)**

Κάθε συνταγογράφηση εκδίδεται υποχρεωτικά από έναν συγκεκριμένο ιατρό (**doctor**), ενώ ένας ιατρός μπορεί να εκδίδει πολλές συνταγογραφήσεις (**prescription**). Συνεπώς, η σχέση αυτή είναι '**1:N**' από **DOCTOR** προς **PRESCRIPTION** με ολική συμμετοχή από την πλευρά του prescription.

- **DRUG-PRESCRIPTION (identifying σχέση-PRESCRIBED)**

Κάθε συνταγογράφηση αφορά υποχρεωτικά ένα συγκεκριμένο φάρμακο (**drug**), ενώ ένα φάρμακο μπορεί να εμφανίζεται σε πολλές συνταγογραφήσεις (**prescription**). Συνεπώς, η σχέση αυτή είναι '**1:N**' από **DRUG** προς **PRESCRIPTION** με ολική συμμετοχή από την πλευρά του prescription.

- **HOSPITALIZATION-PRESCRIPTION (identifying σχέση-BELONGS TO)**

Κάθε συνταγογράφηση εκδίδεται στο πλαίσιο μίας συγκεκριμένης νοσηλείας (**hospitalization**), ενώ μία νοσηλεία μπορεί να περιλαμβάνει πολλές συνταγογραφήσεις (**prescription**). Συνεπώς, η σχέση αυτή είναι '**1:N**' από **HOSPITALIZATION** προς **PRESCRIPTION** με ολική συμμετοχή από την πλευρά του prescription.

- **DRUG-ACTIVE_SUBSTANCE (κανονική σχέση-CONTAIN)**

Κάθε φάρμακο περιέχει μία ή περισσότερες δραστικές ουσίες (**active_substance**), και κάθε δραστική ουσία μπορεί να περιέχεται σε πολλά φάρμακα (**drug**). Συνεπώς, η σχέση αυτή είναι '**N:M**' από **DRUG** προς **ACTIVE_SUBSTANCE** με μερική συμμετοχή και από τις δύο πλευρές.

- **PATIENT-ACTIVE_SUBSTANCE (κανονική σχέση-ALLERGIC TO)**

Κάθε ασθενής μπορεί να έχει αλλεργίες σε πολλές δραστικές ουσίες (**active_substance**), και κάθε δραστική ουσία μπορεί να αποτελεί αλλεργιογόνο για πολλούς ασθενείς (**patient**). Συνεπώς, η σχέση αυτή είναι '**N:M**' από **PATIENT** προς **ACTIVE_SUBSTANCE** με μερική συμμετοχή και από τις δύο πλευρές.

- **HOSPITALIZATION-EVALUATION_HOSPITAL (identifying σχέση-EVAL_HOSP)**

Κάθε αξιολόγηση νοσηλείας ανήκει υποχρεωτικά σε μία συγκεκριμένη ολοκληρωμένη νοσηλεία (**hospitalization**), και κάθε νοσηλεία μπορεί να έχει το πολύ μία αξιολόγηση (**evaluation_hospital**). Συνεπώς, η σχέση αυτή είναι **‘1:1’** από **HOSPITALIZATION** προς **EVALUATION_HOSPITAL**, με ολική συμμετοχή (total participation) από την πλευρά του **evaluation_hospital**.

- **HOSPITALIZATION-EVALUATION_DOCTOR (identifying σχέση-EVAL_DOC)**

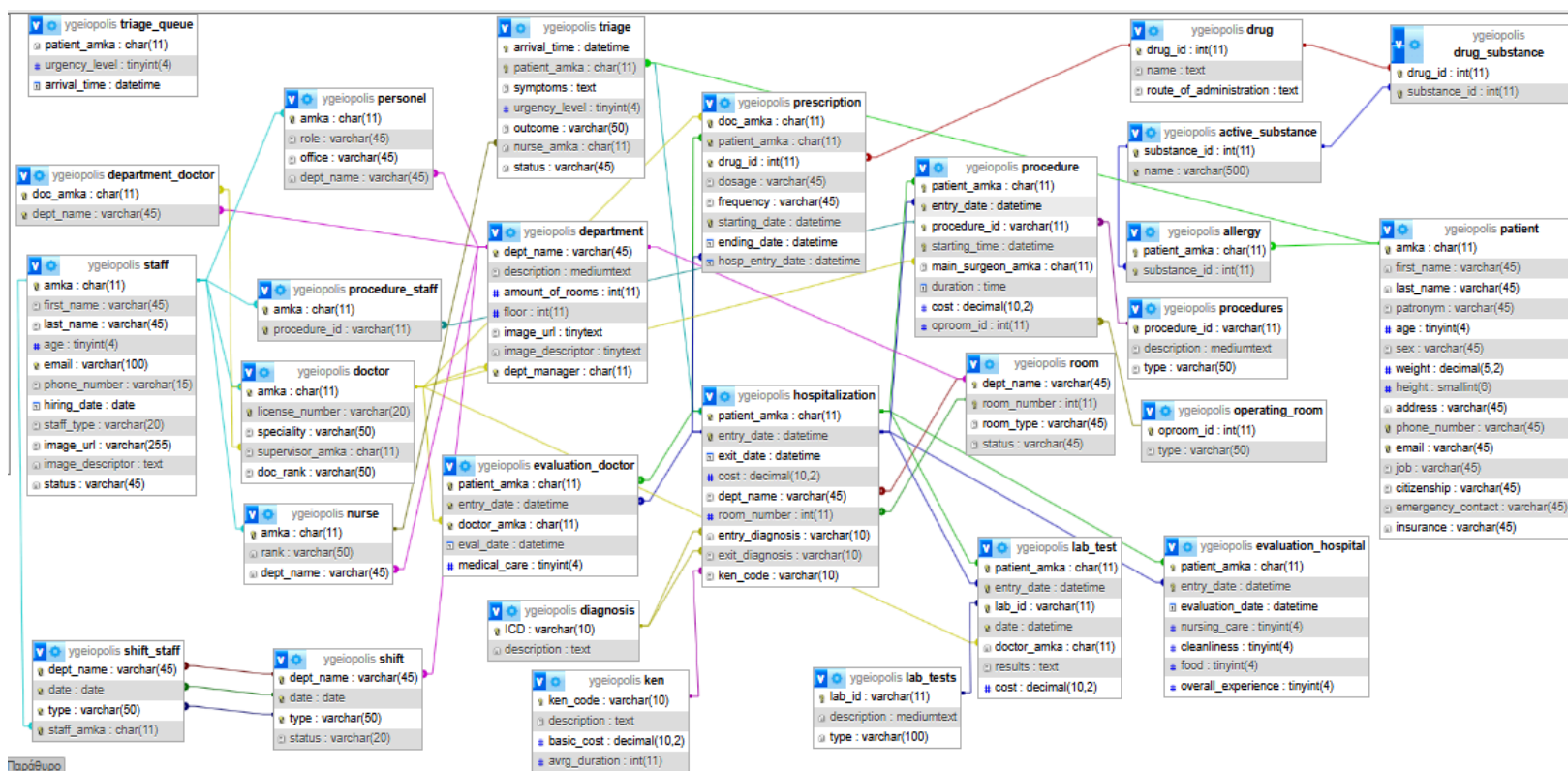
Κάθε αξιολόγηση ιατρού συνδέεται υποχρεωτικά με μία συγκεκριμένη ολοκληρωμένη νοσηλεία (**hospitalization**), και μία νοσηλεία μπορεί να έχει πολλές αξιολογήσεις (**evaluation_doctor**) (μία για κάθε ιατρό). Συνεπώς, η σχέση αυτή είναι **‘1:N’** από **HOSPITALIZATION** προς **EVALUATION_DOCTOR** με ολική συμμετοχή από την πλευρά του **evaluation_doctor**.

- **DOCTOR-EVALUATION_DOCTOR (identifying σχέση-EVALUATION)**

Κάθε αξιολόγηση αφορά υποχρεωτικά έναν συγκεκριμένο ιατρό (**doctor**), ενώ ένας ιατρός μπορεί να έχει πολλές αξιολογήσεις (**evaluation_doctor**) από διαφορετικούς ασθενείς. Συνεπώς, η σχέση αυτή είναι **‘1:N’** από **DOCTOR** προς **EVALUATION_DOCTOR** με ολική συμμετοχή από την πλευρά του **evaluation_doctor**.

Σχεσιακό Διάγραμμα (Relational Model)

Παρακάτω φαίνεται το σχεσιακό διάγραμμα το οποίο κάναμε Reverse Engineer από την βάση δεδομένων μας με την χρήση του PhpMyAdmin το οποίο αναδεικνύει αναλυτικά τους πίνακες και τις στήλες της βάσης:



Κατά τη μετατροπή του Entity Relationship διαγράμματος σε σχεσιακό μοντέλο, οι σχέσεις 1:1 αποδίδονται με την προσθήκη του πρωτεύοντος κλειδιού (primary key) του ενός πίνακα, ως ξένο κλειδί (foreign key) στον άλλον πίνακα. Η επιλογή του πίνακα που θα περιλαμβάνει το ξένο κλειδί εξαρτάται από τη συμμετοχή των οντοτήτων στη σχέση, δηλαδή το total participation, ή από σχεδιαστική απλότητα. Ομοίως, οι σχέσεις 1:N αποδίδονται με την προσθήκη του πρωτεύοντος κλειδιού (primary key) του πίνακα στην πλευρά 1, ως ξένο κλειδί (foreign key) στον πίνακα της πλευράς N. Τέλος, για τις σχέσεις N:N δημιουργείται ένας νέος πίνακας, ο οποίος περιλαμβάνει τα πρωτεύοντα κλειδιά (primary keys) των δυο σχετιζόμενων πινάκων ως ξένα κλειδιά (foreign keys). Ο νέος πίνακας ορίζεται με σύνθετο πρωτεύον κλειδί (composite primary key), το οποίο αποτελείται από τα δύο αυτά attributes, εξασφαλίζοντας τη μοναδικότητα κάθε συσχέτισης.

Περιορισμοί

Constraint CHECK

Ορίσαμε μερικά CHECK CONSTRAINTS σε ορισμένους πίνακες για να τηρήσουμε τα ζητούμενα της εκφώνησης στην ορθή λειτουργία του νοσοκομείου και να θέσουμε περιορισμούς σε λανθασμένες και αντιφατικές εισαγωγές στην βάση. Αυτά τα constraints παρατίθενται παρακάτω:

- **CONSTRAINT `chk\_staff\_type` CHECK (`staff\_type` in ('ΙΑΤΡΟΣ','ΝΟΣΗΛΕΥΤΗΣ','ΔΙΟΙΚΗΤΙΚΟ ΠΡΟΣΩΠΙΚΟ')):**
Στο προσωπικό staff πρέπει να εισάγονται μόνο Ιατροί, Νοσηλευτές και Διοικητικοί Υπάλληλοι.
- **CONSTRAINT `staff\_amka` CHECK (`amka` regexp '^[0-9]{11}\$')**
CONSTRAINT `chk\_patient\_amka` CHECK (`amka` regexp '^[0-9]{11}\$'),
Κάθε άμκα πρέπει να αποτελείται από 11 νούμερα [0-9].
- **CONSTRAINT `chk\_phone\_number` CHECK (`phone\_number` regexp '^+[0-9]{8,15}\$')**
CONSTRAINT `chk\_patient\_phone` CHECK (`phone\_number` regexp '^+[0-9]{8,15}\$')
Κάθε αριθμός τηλεφώνου πρέπει να απαρτίζεται από 8-15 ψηφία ξεκινώντας με '+' ώστε να είναι δεκτοί όλοι οι αριθμοί διεθνώς.
- **CONSTRAINT `chk\_staff\_status` CHECK (`status` in ('ΔΙΑΘΕΣΙΜΟΣ','ΑΠΑΣΧΟΛΗΜΕΝΟΣ'))**
Φροντίζει ώστε η κατάσταση του προσωπικού να παίρνει μόνο τις τιμές 'ΔΙΑΘΕΣΙΜΟΣ' ή 'ΑΠΑΣΧΟΛΗΜΕΝΟΣ' για να είναι εφικτός ο έλεγχος με triggers που θα εξηγηθούν παρακάτω, για το αν είναι διαθέσιμο ένα μέλος του προσωπικού να του ανατεθεί κάποιο πόστο μες το νοσοκομείο.
- **CONSTRAINT `chk\_doctor\_rank` CHECK (`doc\_rank` in ('ΕΙΔΙΚΕΥΟΜΕΝΟΣ','ΕΠΙΜΕΛΗΤΗΣ Β','ΕΠΙΜΕΛΗΤΗΣ Α','ΔΙΕΥΘΥΝΤΗΣ'))**
Περιορίζει τα επίπεδα βαθμίδας ενός ιατρού στα ζητούμενα για την ορθή λειτουργία και οργάνωση του νοσοκομείου.
- **CONSTRAINT `chk\_nurse\_rank` CHECK (`rank` in ('ΒΟΗΘΟΣ ΝΟΣΗΛΕΥΤΗ','ΝΟΣΗΛΕΥΤΗΣ','ΠΡΟΪΣΤΑΜΕΝΟΣ'))**
Ομοίως και με την βαθμίδα του νοσηλευτή
- **CONSTRAINT `chk\_role` CHECK (`role` in ('ΓΡΑΜΜΑΤΕΑΣ ΤΜΗΜΑΤΟΣ','ΥΠΕΥΘΥΝΟΣ ΑΡΧΕΙΟΥ','ΔΙΑΧΕΙΡΙΣΤΗΣ ΡΑΝΤΕΒΟΥ','ΥΠΕΥΘΥΝΟΣ ΕΙΣΑΓΩΓΩΝ','ΛΟΓΙΣΤΗΣ','ΥΠΕΥΘΥΝΟΣ ΠΡΟΜΗΘΕΙΩΝ','ΔΙΟΙΚΗΤΙΚΟΣ ΣΥΝΤΟΝΙΣΤΗΣ','ΥΠΕΥΘΥΝΟΣ ΑΣΦΑΛΙΣΕΩΝ'))**
Ομοίως ορίσαμε μερικούς αυτοσχέδιους ρόλους για το Διοικητικό προσωπικό που θεωρήσαμε απαραίτητους για την πιο γραφειοκρατική λειτουργία του νοσοκομείου.

- **CONSTRAINT `chk\_room\_type` CHECK (`room\_type` in ('ΠΟΛΥΚΛΙΝΟ','ΔΙΚΛΙΝΟ','ΜΟΝΟΚΛΙΝΟ','ΜΕΘ'))**
Εδώ ορίσαμε τις επιλογές χωρητικότητας μιας κλίνης, κάτι το οποίο θα παίξει πολύ σημαντικό ρόλο στην εύρεση πληρότητας και διαθεσιμότητας αργότερα.
- **CONSTRAINT `chk\_room\_status` CHECK (`status` in ('ΚΑΤΕΙΛΛΗΜΕΝΟ','ΔΙΑΘΕΣΙΜΟ','ΥΠΟ ΣΥΝΤΗΡΗΣΗ'))**
Για να ορίσουμε λοιπόν αυτή τη διαθεσιμότητα θέσαμε τις παραπάνω καταστάσεις μιας κλίνης.
- **CONSTRAINT `chk\_patient\_insurance` CHECK (`insurance` in ('Ανασφάλιστος','ΕΟΠΥΥ','ΕΦΚΑ','Ιδιωτική Ασφάλεια'))**
Ορίζει τις επιλογές ασφαλιστικού φορέα.
- **CONSTRAINT `chk\_shift\_type` CHECK (`type` in ('ΠΡΩΙΝΗ','ΑΠΟΓΕΥΜΑΤΙΝΗ','ΝΥΧΤΕΡΙΝΗ'))**
Ορίζει τις τρεις βάρδιες της κάθε ημέρας έτσι όπως μας ζητήθηκε
- **CONSTRAINT `chk\_status` CHECK (`status` in ('ΜΗ ΕΓΚΥΡΗ','ΕΓΚΥΡΗ'))**
Αυτό το constraint είναι πολύ σημαντικό γιατί καθορίζει τα δύο στάδια μιας βάρδιας προτού ολοκληρωθεί ο προγραμματισμός της. Ύστερα θα μας ζητηθούν συγκεκριμένοι όροι για την κάθε βάρδια και μόνο όταν πληρούνται αυτοί οι όροι, θα τεθεί ως έγκυρη η βάρδια.
- **CONSTRAINT `chk\_urgency\_level` CHECK (`urgency\_level` in ('1','2','3','4','5'))**
Ορίζει τα 5 επίπεδα επείγοντος που θα αναθέσει ο νοσηλευτής διαλογής σε κάθε ασθενή που καταφθάνει στα επείγοντα.
- **CONSTRAINT `chk\_status` CHECK (`status` in ('ΑΝΑΜΟΝΗ','ΕΞΕΤΑΖΕΤΑΙ','ΟΛΟΚΛΗΡΩΘΗΚΕ'))**
Το status είναι η κατάσταση της κάθε διαλογής, η οποία μας βοηθάει να δούμε το στάδιο που βρίσκεται ο κάθε ασθενής κατά την διαλογή. Στην ΑΝΑΜΟΝΗ περιμένει έναν νοσηλευτή να τον αναλάβει, όσο ΕΞΕΤΑΖΕΤΑΙ ο νοσηλευτής καθιστάται ως απασχολημένος -όπως αναφέραμε και πριν- βγάζοντας ένα αποτέλεσμα για τον ασθενή και όταν ΟΛΟΚΛΗΡΩΘΗΚΕ, ο νοσηλευτής ξανά είναι διαθέσιμος και ο ασθενής θα σταλεί είτε για νοσηλεία είτε προς αποχώρηση.
- **CONSTRAINT `chk\_outcome` CHECK (`outcome` in ('ΝΟΣΗΛΕΙΑ','ΑΠΟΧΩΡΗΣΗ'))**
Και εδώ ορίζουμε το αν θα σταλεί για νοσηλεία ή αποχώρηση όπως αναφέρθηκε
- **CONSTRAINT `chk\_op\_room\_type` CHECK (`type` in ('ΧΕΙΡΟΥΡΓΕΙΟ','ΑΙΘΟΥΣΑ ΕΠΕΜΒΑΣΗΣ'))**
Εδώ ορίζονται οι δύο χώροι επεμβάσεων
- Τέλος στα παρακάτω constraint περιορίζονται τα άκρα κάθε αξιολόγησης μεταξύ 1 έως 5 ώστε να τηρηθεί η κλίμακα Likert που μας ζητείται.
CHECK (`medical\_care` between 1 and 5)
CHECK (`nursing\_care` between 1 and 5)
CHECK (`cleanliness` between 1 and 5)
CHECK (`food` between 1 and 5)
CHECK (`overall\_experience` between 1 and 5)

Triggers

Triggers Πίνακα doctor (Ιατρικό Προσωπικό)

Για τη σωστή διαχείριση των ιατρών και τη διασφάλιση της συνοχής των δεδομένων σε σχέση με την ιεραρχία, την υποχρεωτική εποπτεία και την αποτροπή κυκλικών αναφορών, υλοποιήθηκαν δύο (2) triggers:

1. trg_doctor_before_insert

- **Χρονισμός:** Ενεργοποιείται πριν από την εισαγωγή κάθε νέας εγγραφής ιατρού στον πίνακα.
- **Λειτουργικότητα:** Αρχικά ελέγχει αν ο βαθμός του νέου ιατρού είναι 'ΕΙΔΙΚΕΥΟΜΕΝΟΣ', απαιτώντας υποχρεωτικά την ύπαρξη επόπτη, και αν ο βαθμός είναι 'ΔΙΕΥΘΥΝΤΗΣ', απαγορεύει τον ορισμό επόπτη. Επίσης, εξετάζει αναδρομικά όλη την αλυσίδα των εποπτών για να διασφαλίσει ότι δεν δημιουργείται κυκλική αλυσίδα εποπτείας. Σε περίπτωση παραβίασης, η εισαγωγή διακόπτεται με μήνυμα σφάλματος, γραμμένο κατάλληλα για την κάθε περίπτωση.

2. doctor_BEFORE_UPDATE

- **Χρονισμός:** Ενεργοποιείται πριν από την τροποποίηση μιας ήδη υπάρχουσας εγγραφής ιατρού.
 - **Λειτουργικότητα:** Λειτουργεί ως δικλείδα ασφαλείας κατά την ανανέωση των δεδομένων. Εκτελεί τους ίδιους ακριβώς ελέγχους με τον trigger **trg_doctor_before_insert**. Αν διαπιστωθεί ότι με την τροποποίηση υπάρξη παραβίαση, η διαδικασία ακυρώνεται αμέσως και επιστρέφεται προειδοποιητικό μήνυμα.
-

Triggers Πίνακα room (Κλίνες / Δωμάτια)

Για τη σωστή διαχείριση των δωματίων και τη διασφάλιση της συνοχής των δεδομένων σε σχέση με τα νοσοκομειακά τμήματα και τις τρέχουσες νοσηλείες, υλοποιήθηκαν τρία (3) triggers:

1. department_AFTER_INSERT

- **Χρονισμός:** Ενεργοποιείται μετά από την επιτυχή εισαγωγή μιας νέας κλίνης στον πίνακα.
- **Λειτουργικότητα:** Αναλαμβάνει την αυτόματη ενημέρωση των *amount_of_rooms* των αντίστοιχων *departments*. Συγκεκριμένα, αυξάνει κατά ένα (+1) την τιμή του συνολικού αριθμού δωματίων στον πίνακα του τμήματος στο οποίο ανήκει η νέα κλίνη.

2. room_BEFORE_DELETE

- **Χρονισμός:** Ενεργοποιείται πριν από τη διαγραφή μιας κλίνης.
- **Λειτουργικότητα:** Εξετάζει τον πίνακα των νοσηλειών και εξετάζει αν υπάρχουν ενεργά περιστατικά στο συγκεκριμένο δωμάτιο. Αν βρεθεί έστω και μία ενεργή νοσηλεία, η διαγραφή μπλοκάρεται αμέσως και επιστρέφεται μήνυμα ότι το δωμάτιο είναι κατειλημμένο.

3. department_AFTER_DELETE

- **Χρονισμός:** Ενεργοποιείται μετά από την επιτυχή διαγραφή μιας κλίνης.
 - **Λειτουργικότητα:** Λειτουργεί αντίστροφα από τον trigger **department_AFTER_INSERT**. Μειώνει κατά ένα (-1) την τιμή του συνολικού αριθμού δωματίων στον πίνακα του τμήματος στο οποίο ανήκει η νέα κλίνη.
-

Triggers Πίνακα shift_staff (Συμμετοχή Προσωπικού σε Βάρδιες)

Για την αυστηρή τήρηση των κανόνων ασφαλείας, των εργασιακών ορίων του νοσοκομείου και τη διασφάλιση της ορθής στελέχωσης των εφημεριών, υλοποιήθηκαν πέντε (5) triggers:

1. chk_staff_department_before_insert

- **Χρονισμός:** Ενεργοποιείται πριν από την εισαγωγή κάθε νέας συμμετοχής υπαλλήλου σε βάρδια.
- **Λειτουργικότητα:** Ανακτά την ιδιότητα του υπαλλήλου (Ιατρός, Νοσηλεύτης, ή Διοικητικός) και ελέγχει αν ο συγκεκριμένος εργαζόμενος είναι πράγματι εγγεγραμμένος στο τμήμα όπου πρόκειται να εκτελεστεί η βάρδια. Αν δεν ανήκει σε αυτό, η καταχώρηση απαγορεύεται.

2. trg_insert_shift_staff_intern_supervision

- **Χρονισμός:** Ενεργοποιείται πριν από την εισαγωγή μιας νέας εγγραφής ιατρού σε βάρδια.
- **Λειτουργικότητα:** Όταν επιχειρείται η καταχώρηση ενός ειδικευόμενου ιατρού, ο trigger εξετάζει τις υπόλοιπες συμμετοχές στη συγκεκριμένη βάρδια. Αν διαπιστωθεί ότι δεν συμμετέχει κανένας ανώτερος γιατρός (Επιμελητής ή Διευθυντής), η εισαγωγή ακυρώνεται, αφού απαγορεύεται η εφημερία ειδικευόμενων χωρίς επίβλεψη.

3. trg_shift_limits

- **Χρονισμός:** Ενεργοποιείται πριν από την εισαγωγή οποιουδήποτε μέλους προσωπικού σε βάρδια.
- **Λειτουργικότητα:** Αυτός ο trigger ελέγχει τρεις βασικούς περιορισμούς:
 - *Μηνιαία Όρια:* Ελέγχει τις μηνιαίες βάρδιες του υπαλλήλου και μπλοκάρει την καταχώρηση αν ξεπεραστεί το ανώτατο όριο (15 για ιατρούς, 20 για νοσηλευτές, 25 για διοικητικούς).
 - *Ελάχιστη Ανάπαυση:* Εξετάζει τις διαδοχικές βάρδιες, διασφαλίζοντας ότι μεσολαβεί το απαραίτητο 8ωρο ξεκούρασης.
 - *Περιορισμός Νυχτερινών:* Σε περίπτωση νυχτερινής βάρδιας, εξετάζει τις 'γειτονικές' ημερομηνίες και εμποδίζει την καταχώρηση αν ο υπάλληλος συμπληρώνει περισσότερες από 3 συνεχόμενες νύχτες εργασίας.

4. trg_shift_staff_after_insert_status

- **Χρονισμός:** Ενεργοποιείται μετά από την επιτυχή εισαγωγή μιας νέας συμμετοχής υπαλλήλου σε βάρδια.
- **Λειτουργικότητα:** Κάθε φορά που προστίθεται ένας εργαζόμενος, ο trigger υπολογίζει το συνολικό πλήθος ιατρών, νοσηλευτών και διοικητικών που έχουν δηλωθεί για τη συγκεκριμένη βάρδια. Μόλις ικανοποιηθούν τα ελάχιστα όρια ασφαλείας (τουλάχιστον 3 γιατροί, 6 νοσηλευτές και 2 διοικητικοί), ενημερώνει αυτόματα την κατάσταση της βάρδιας στον πίνακα shift ως 'έγκυρη'.

5. trg_delete_shift_staff_intern_supervision

- **Χρονισμός:** Ενεργοποιείται πριν από τη διαγραφή μιας εγγραφής από τον πίνακα βαρδιών.
- **Λειτουργικότητα:** Εάν επιχειρηθεί η αφαίρεση ενός ανώτερου ιατρού από μια βάρδια στην οποία συμμετέχουν ειδικευόμενοι, ο trigger ελέγχει αν ο γιατρός αυτός είναι ο τελευταίος διαθέσιμος επόπτης. Αν είναι ο μοναδικός, η διαγραφή μπλοκάρεται για να μην μείνουν οι ειδικευόμενοι χωρίς επίβλεψη.

Triggers Πίνακα triage (Διαλογή Επειγόντων Περιστατικών)

Για τη σωστή διαχείριση της ροής των επειγόντων περιστατικών, την παρακολούθηση της κατάστασης των ασθενών και την αυτόματη διαχείριση της διαθεσιμότητας του νοσηλευτικού προσωπικού, υλοποιήθηκαν τρεις (3) triggers:

1. triage_status_check

- **Χρονισμός:** Ενεργοποιείται πριν από την εισαγωγή κάθε νέας εγγραφής περιστατικού στον πίνακα διαλογής.
- **Λειτουργικότητα:** Ελέγχει τη διαθεσιμότητα του προσωπικού. Αν η κατάσταση του ασθενούς ορίζεται άμεσα ως 'ΕΞΕΤΑΖΕΤΑΙ', ο trigger εξετάζει αν ο υπεύθυνος νοσηλευτής είναι ελεύθερος. Εάν εμφανίζεται ως 'ΑΠΑΣΧΟΛΗΜΕΝΟΣ', η εισαγωγή διακόπτεται.

2. triage_status_occupy

- **Χρονισμός:** Ενεργοποιείται μετά από την επιτυχή εισαγωγή μιας νέας εγγραφής περιστατικού.
- **Λειτουργικότητα:** Εάν ο ασθενής εισαχθεί με κατάσταση 'ΕΞΕΤΑΖΕΤΑΙ', ο trigger ενημερώνει αυτόματα τον πίνακα του προσωπικού, μεταβάλλοντας την κατάσταση του αντίστοιχου νοσηλευτή σε 'ΑΠΑΣΧΟΛΗΜΕΝΟΣ'.

3. triage_status_change

- **Χρονισμός:** Ενεργοποιείται μετά από την τροποποίηση της κατάστασης ενός υπάρχοντος περιστατικού διαλογής.
 - **Λειτουργικότητα:** Παρακολουθεί την πορεία του ασθενούς και ενημερώνει την διαθεσιμότητα του νοσηλευτή:
 - *Έναρξη Εξέτασης:* Όταν ο ασθενής μεταβαίνει από την 'ΑΝΑΜΟΝΗ' στο 'ΕΞΕΤΑΖΕΤΑΙ', ο trigger δεσμεύει τον νοσηλευτή αλλάζοντας την κατάστασή του σε 'ΑΠΑΣΧΟΛΗΜΕΝΟΣ'.
 - *Ολοκλήρωση:* Όταν η κατάσταση μεταβεί σε 'ΟΛΟΚΛΗΡΩΘΗΚΕ', ο trigger αποδεσμεύει αυτόματα τον νοσηλευτή, επαναφέροντας την κατάστασή του σε 'ΔΙΑΘΕΣΙΜΟΣ' για το επόμενο περιστατικό.
-

Triggers Πίνακα hospitalization (Νοσηλείες Ασθενών)

Για την αυτοματοποιημένη διαχείριση των κλινών, τον έλεγχο πληρότητας των δωματίων και τον ακριβή υπολογισμό του κόστους νοσηλείας βάσει των Κλειστών Ενοποιημένων Νοσηλίων (KEN), υλοποιήθηκαν τέσσερις (4) triggers:

1. hosp_bef_ins_chk_status

- **Χρονισμός:** Ενεργοποιείται πριν από την εισαγωγή κάθε νέας εγγραφής νοσηλείας.
- **Λειτουργικότητα:** Αποτρέπει την εισαγωγή ασθενών σε μη διαθέσιμα δωμάτια εκτελώντας δύο ελέγχους:
 - *Κατάσταση:* Ακυρώνει τη διαδικασία αν το δωμάτιο βρίσκεται σε κατάσταση 'ΥΠΟ ΣΥΝΤΗΡΗΣΗ'.
 - *Πληρότητα:* Αναγνωρίζει τη μέγιστη χωρητικότητα βάσει του τύπου του δωματίου και μετρά τους ήδη νοσηλευόμενους. Αν το δωμάτιο είναι πλήρες, απαγορεύει τη νέα εισαγωγή.

2. hosp_aft_ins_upd_status

- **Χρονισμός:** Ενεργοποιείται μετά από την επιτυχή εισαγωγή μιας νέας νοσηλείας.
- **Λειτουργικότητα:** Συγχρονίζει την κατάσταση των κλινών σε πραγματικό χρόνο. Αμέσως μετά την τοποθέτηση του ασθενούς, επανυπολογίζει το πλήθος των νοσηλευόμενων στο δωμάτιο. Εάν ο αριθμός αγγίξει τη μέγιστη χωρητικότητα, τροποποιεί αυτόματα την κατάσταση του δωματίου σε 'ΠΛΗΡΕΣ'.

3. hosp_bef_upd_exit_date

- **Χρονισμός:** Ενεργοποιείται πριν από την οριστικοποίηση μιας τροποποίησης στον πίνακα νοσηλειών.
- **Λειτουργικότητα:** Αποτελεί τον μηχανισμό τιμολόγησης κατά το εξιτήριο του ασθενούς. Μόλις καταχωρηθεί η ημερομηνία εξόδου, ο trigger ανακτά το βασικό κόστος και τη μέση διάρκεια από τον πίνακα των ΚΕΝ:
 - **Υπολογισμός:** Υπολογίζει τις πραγματικές ημέρες νοσηλείας. Εάν ο ασθενής ξεπεράσει τη προβλεπόμενη μέση διάρκεια, ο trigger επιβάλλει επιπλέον χρέωση για κάθε επιπλέον ημέρα βάσει του ημερήσιου κόστους. Διαφορετικά, χρεώνεται αποκλειστικά το βασικό ποσό του ΚΕΝ.

4. hospitalization_AFTER_UPDATE

- **Χρονισμός:** Ενεργοποιείται μετά από την ολοκλήρωση μιας τροποποίησης στον πίνακα νοσηλειών.
 - **Λειτουργικότητα:** Διαχειρίζεται την αποδέσμευση των κλινών. Εάν η τροποποίηση αφορά την έκδοση εξιτηρίου, ο trigger καταμετρά εκ νέου τους εναπομείναντες ασθενείς στο δωμάτιο και επαναφέρει την κατάσταση του δωματίου σε 'ΔΙΑΘΕΣΙΜΟ', επιτρέποντας μελλοντικές εισαγωγές.
-

Triggers Πίνακα lab_test (Εργαστηριακές Εξετάσεις)

Για την παρακολούθηση της ροής των εργαστηριακών εξετάσεων και τη δυναμική διαχείριση της διαθεσιμότητας του ιατρικού προσωπικού που τις επιβλέπει, υλοποιήθηκαν τρεις (3) triggers:

1. lab_test_status_check

- **Χρονισμός:** Ενεργοποιείται πριν από την εισαγωγή κάθε νέας εγγραφής εξέτασης.
- **Λειτουργικότητα:** Διασφαλίζει ότι δεν ανατίθενται νέες εκκρεμείς εξετάσεις σε γιατρούς που είναι ήδη φορτωμένοι. Εάν η νέα εξέταση εισάγεται χωρίς ακόμα να έχει αποτέλεσμα και ο επιλεγμένος ιατρός εμφανίζεται ως 'ΑΠΑΣΧΟΛΗΜΕΝΟΣ', ο trigger διακόπτει την εισαγωγή για να αποτρέψει την υπερφόρτωση του γιατρού.

2. lab_test_status_occupy

- **Χρονισμός:** Ενεργοποιείται μετά από την επιτυχή εισαγωγή μιας νέας εργαστηριακής εξέτασης.
- **Λειτουργικότητα:** Ενημερώνει την κατάσταση του ιατρικού προσωπικού κατά την έναρξη της διαδικασίας. Μόλις καταχωρηθεί μια εξέταση που τελεί υπό έκδοση αποτελέσματος, ο trigger μεταβάλλει αυτόματα την κατάσταση του αντίστοιχου ιατρού σε 'ΑΠΑΣΧΟΛΗΜΕΝΟΣ'.

3. lab_test_status_free

- **Χρονισμός:** Ενεργοποιείται μετά από την τροποποίηση μιας ήδη υπάρχουσας εργαστηριακής εξέτασης.
 - **Λειτουργικότητα:** Λειτουργεί ως μηχανισμός αυτόματης αποδέσμευσης. Παρακολουθεί τη μετάβαση των δεδομένων και ενεργοποιείται αποκλειστικά τη στιγμή που καταχωρούνται τα αποτελέσματα της εξέτασης. Καθώς το εκκρεμές έργο ολοκληρώθηκε, επαναφέρει την κατάσταση του ιατρού σε 'ΔΙΑΘΕΣΙΜΟΣ'.
-

Triggers Πίνακα procedure (Ιατρικές Πράξεις / Επεμβάσεις)

Για τη διασφάλιση της ορθότητας του προγραμματισμού των χειρουργικών επεμβάσεων, την αποφυγή χρονικών συμπτώσεων και τη δυναμική διαχείριση της διαθεσιμότητας των χειρουργών, υλοποιήθηκαν τρεις (3) triggers:

1. procedure_BEFORE_INSERT

- **Χρονισμός:** Ενεργοποιείται πριν από την εισαγωγή κάθε νέας προγραμματισμένης επέμβασης.
- **Λειτουργικότητα:** Αποτρέπει τις διπλοκρατήσεις και τις χρονικές επικαλύψεις μέσω δύο ελέγχων:
 - *Χώρος:* Ελέγχει αν η επιλεγμένη αίθουσα χειρουργείου είναι ήδη κατειλημμένη κατά το αιτούμενο χρονικό διάστημα. Σε περίπτωση σύγκρουσης, εγείρει σφάλμα.
 - *Ιατρός:* Ελέγχει αν ο κύριος χειρουργός έχει αναλάβει άλλη επέμβαση που βρίσκεται σε εξέλιξη το ίδιο χρονικό διάστημα, εμποδίζοντας την ταυτόχρονη απασχόλησή του σε δύο χειρουργεία.

2. procedure_status_check

- **Χρονισμός:** Ενεργοποιείται πριν από την εισαγωγή της εγγραφής στον πίνακα επεμβάσεων.
- **Λειτουργικότητα:** Προστατεύει τη διαθεσιμότητα του ιατρικού προσωπικού. Εάν ο χειρουργός εμφανίζεται ήδη ως 'ΑΠΑΣΧΟΛΗΜΕΝΟΣ' και η νέα επέμβαση αφορά χρονικό διάστημα που εκτείνεται στο παρόν ή στο μέλλον, ο trigger απαγορεύει την καταχώρηση.

3. procedure_status_occupy

- **Χρονισμός:** Ενεργοποιείται μετά από την επιτυχή εισαγωγή μιας νέας επέμβασης.
 - **Λειτουργικότητα:** Συγχρονίζει την κατάσταση του ιατρού σε πραγματικό χρόνο ανάλογα με την ώρα. Μεταβάλλει την κατάσταση του χειρουργού σε 'ΑΠΑΣΧΟΛΗΜΕΝΟΣ' μόνο υπό την προϋπόθεση ότι η επέμβαση ξεκινά άμεσα (ή έχει ήδη ξεκινήσει) και δεν έχει παρέλθει η ώρα ολοκλήρωσής της. Αν η επέμβαση αφορά μελλοντικό χρόνο, ο ιατρός παραμένει 'ΔΙΑΘΕΣΙΜΟΣ'.
-

Triggers Πίνακα prescription (Συνταγογράφηση Φαρμακευτικής Αγωγής)

Για την εξάλειψη του ιατρικού λάθους και την απόλυτη διασφάλιση της υγείας των νοσηλευόμενων, υλοποιήθηκε ένας κρίσιμος trigger ελέγχου:

1. trg_allergy_substance

- **Χρονισμός:** Ενεργοποιείται πριν από την εισαγωγή κάθε νέας φαρμακευτικής συνταγής στον πίνακα.
- **Λειτουργικότητα:** Απαγορεύει τη συνταγογράφηση φαρμάκων στα οποία ο ασθενής παρουσιάζει αλλεργία. Καθώς ένα φάρμακο μπορεί να αποτελείται από περισσότερες από μία δραστικές ουσίες, ο trigger ελέγχει τα δεδομένα σε επίπεδο χημικών συστατικών. Πραγματοποιεί διασταύρωση ανάμεσα στον πίνακα που συνδέει τα φάρμακα με τις ουσίες τους και στον πίνακα όπου καταγράφονται οι διαπιστωμένες αλλεργίες του ασθενούς βάσει του ΑΜΚΑ του. Εάν βρεθεί έστω και μία δραστική ουσία που προκαλεί αλλεργία, ο trigger μπλοκάρει άμεσα τη συνταγογράφηση και εκδίδει μήνυμα σφάλματος.

Ευρετήρια (Indexes)

Τα ευρετήρια προστέθηκαν στη βάση δεδομένων για να βελτιστοποιήσουν την ταχύτητα εκτέλεσης των ερωτημάτων (queries), θέτοντας σε κάθε πίνακα μία διάταξη που επιλέξαμε εμείς για να οργανωθούν ομάλα όλα τα στοιχεία σε κάθε πίνακα. Θα τονίσουμε ότι με την χρήση του MySQL Workbench όπου βασίσαμε την εργασία μας, μας βοήθησε πολύ το interface, καθώς με κάθε προσθήκη foreign key δημιουργήθηκε και από ένα index πέρα από το ήδη υπάρχον PRIMARY index που υπάρχει σε κάθε πίνακα και κατατάσσει με βάση το εκάστοτε id. Συγκεκριμένα χρησιμοποιήσαμε κυρίως τα παρακάτω:

- **create index idx_eval_doc_amka on evaluation_doctor (doctor_amka);**

Στο ερώτημα Q4 (Για συγκεκριμένο ιατρό,...) αυτό το ευρετήριο είναι απαραίτητο καθώς φιλτράρουμε τον πίνακα **evaluation_doctor** βάσει συγκεκριμένου ιατρού (**WHERE ed.doctor_amka = '...'**). Χωρίς το ευρετήριο ο optimizer εκτελεί full table scan, ενώ με την παρουσία του επιλέγει αυτόματα index lookup (type=ref).

- **create index if not exists idx_hosp_patient on hospitalization (patient_amka);**

Στο ερώτημα Q6 (Για συγκεκριμένο ασθενή,...) αυτό το ευρετήριο είναι απαραίτητο καθώς φιλτράρουμε τον πίνακα **hospitalization** βάσει συγκεκριμένου ασθενή (**WHERE h.patient_amka = '...'**). Χωρίς το ευρετήριο ο optimizer εκτελεί full table scan, ενώ με την παρουσία του επιλέγει αυτόματα index lookup (type=ref).

- **create index if not exists idx_eval_hosp_patient on evaluation_hospital (patient_amka, entry_date);**

Στο ερώτημα Q6 (Για συγκεκριμένο ασθενή,...) αυτό το ευρετήριο είναι απαραίτητο καθώς χρησιμοποιείται στο **JOIN** μεταξύ **hospitalization** και **evaluation_hospital** βάσει του συνδυασμού (**ON h.patient_amka = eh.patient_amka AND h.entry_date = eh.entry_date**). Χωρίς το ευρετήριο ο optimizer εκτελεί full table scan, ενώ με την παρουσία του επιλέγει αυτόματα index lookup (type=ref) χρησιμοποιώντας και τα δύο πεδία του composite index ταυτόχρονα.

Views

Με την χρήση των View, έχουμε φτιάξει έναν εικονικό πίνακα που μας βοηθάει να δείξουμε ανά πάσα στιγμή την κατάσταση της διαλογής και συγκεκριμένα μας δείχνει την “ουρά αναμονής”.

```
CREATE ALGORITHM=UNDEFINED DEFINER=`ygeiopolis_team`@`%` SQL SECURITY
DEFINER VIEW `triage_queue` AS
SELECT `triage`.`patient_amka` AS `patient_amka`,
`triage`.`urgency_level` AS `urgency_level`,
`triage`.`arrival_time` AS `arrival_time`
FROM `triage`
WHERE `triage`.`status` = 'ANAMONH'
ORDER BY `triage`.`urgency_level`, `triage`.`arrival_time`
```

Στην ουσία επιλέγει από τον πίνακα **triage** όλα τα αμκα των ασθενών που βρίσκονται σε κατάσταση ‘ANAMONH’ και τους βάζει στην σειρά προτεραιότητας που μας ζητήθηκε. Πρώτα προηγείται το επίπεδο επείγοντος και έπειτα ο χρόνος άφιξης. Έτσι βλέπουμε έναν έναν τους ασθενείς στην αναμονή και ξέρουμε ποιος είναι ο επόμενος που πρέπει να του τεθεί ένας νοσηλευτής για να τον αναλάβει.

Events

Με την χρήση των Events φτιάξαμε έναν “μηχανισμό” που ελέγχει συνεχώς αν υπάρχει κάποιο μέλος του προσωπικού που δεν είναι πια απασχολημένο σε κάποιο πόστο του Νοσοκομείου.

```

CREATE DEFINER=`ygeiopolis_team`@`%` EVENT `free_all_staff_timer` ON
SCHEDULE EVERY 1 MINUTE STARTS '2026-05-13 01:55:29' ON COMPLETION
NOT PRESERVE ENABLE DO BEGIN
    UPDATE `staff`
    SET `status` = 'ΔΙΑΘΕΣΙΜΟΣ'
    WHERE `status` = 'ΑΠΑΣΧΟΛΗΜΕΝΟΣ'

    AND amka NOT IN (
        SELECT main_surgeon_amka
        FROM `procedure`
        WHERE starting_time <= now() AND addtime(starting_time,
duration) > now()
    )

    AND amka NOT IN (
        SELECT ps.amka
        FROM `procedure_staff` ps
        JOIN `procedure` p ON ps.procedure_id = p.procedure_id
        WHERE p.starting_time <= now() AND addtime(p.starting_time,
p.duration) > now()
    )

    AND amka NOT IN (
        SELECT doctor_amka
        FROM `lab_test`
        WHERE results IS NULL
    )

    AND amka NOT IN (
        SELECT nurse_amka
        FROM `triage`
        WHERE status != 'ΟΛΟΚΛΗΡΩΘΗΚΕ'
    );
END

```

Συγκεκριμένα, κάθε ένα λεπτό γίνεται έλεγχος στις 3 διαδικασίες που απασχολούν προσωπικό. Αρχικά ελέγχουμε αν έχει τελειώσει κάποιο procedure ώστε να θέσουμε τον χειρουργό και τους βοηθούς που δούλευαν εκεί ως διαθέσιμους, καθώς δεν έχουμε βάλει κάποιο ending time για να κάνουν update τα trigger. Ομοίως αν και έχουμε βάλει trigger, θεωρήσαμε καλό να προσθέσουμε και τον ίδιο έλεγχο στα lab_test και στην διαλογή. Έτσι η βάση μας είναι 100% ασφαλή ότι δεν θα ξεχαστεί κάποιος σε κάποιο πόστο ως Απασχολημένος.

Fake Data

Για την προσθήκη δεδομένων και το γέμισμα της βάσης χρησιμοποιήσαμε την εφαρμογή [Cobbli.io](https://cobbli.io) που μας δόθηκε στις διαφάνειες κάνοντας generate όλα τα ονόματα, τις ηλικίες, τα email, τα τηλέφωνα, τα αμκα, τα ύψη, τα βάρη, τις διευθύνσεις των κατοικιών, τις εργασίες και τις εθνικότητες. Έτσι γεμίσαμε τα tables staff και patient. Έπειτα έγινε αρκετή χρήση του Gemini LLM το οποίο χρησιμοποιήσαμε για να μας βοηθήσει να φτιάξουμε python scripts τα οποία συνδέσαμε με τον Host της βάσης ώστε να χρησιμοποιηθούν ήδη υπάρχοντα άμκα και δεδομένα για να γεμίσουν και οι υπόλοιποι πίνακες της βάσης, καθώς είχαμε ελάχιστη γνώση της γλώσσας. Επίσης για την φόρτωση των excel που μας δόθηκαν, χρησιμοποιήσαμε το site https://www.convertcsv.com/csv-to-sql.htm#google_vignette στο οποίο στέλναμε τα αρχεία αφού τα μετατρέψαμε σε CSV και μας τα μετέτρεψε σε εντολές INSERT. Τέλος κάναμε export όλα τα δεδομένα στο load.sql που υποβλήθηκε.

Queries

Q1: Βρείτε τα συνολικά έσοδα του νοσοκομείου ανά τμήμα και ανά έτος, με ανάλυση ανά KEN κωδικό (βασικό κόστος έναντι πρόσθετης χρέωσης λόγω υπέρβασης ΜΑΝ) και κατανομή νοσηλειών ανά ασφαλιστικό φορέα.

```
WITH hosp_costs AS (  
  SELECT  
    h.dept_name,  
    YEAR(IFNULL(h.exit_date, h.entry_date)) AS eval_year,  
    h.ken_code,  
    p.insurance,  
    h.patient_amka,  
    h.entry_date,  
    k.basic_cost AS base_ken_cost,
```

```

        h.cost AS total_hosp_cost,
        (h.cost - k.basic_cost) AS extra_ken_cost
FROM `hospitalization` h
JOIN `patient` p ON h.patient_amka = p.amka
JOIN `ken` k ON h.ken_code = k.ken_code
WHERE h.cost IS NOT NULL
),
proc_costs AS (
    SELECT patient_amka, entry_date, SUM(cost) AS total_proc_cost
    FROM `procedure`
    GROUP BY patient_amka, entry_date
),
lab_costs AS (
    SELECT patient_amka, entry_date, SUM(cost) AS total_lab_cost
    FROM `lab_test`
    GROUP BY patient_amka, entry_date
)
SELECT
    hc.dept_name AS 'Τμήμα',
    hc.eval_year AS 'Έτος',
    hc.ken_code AS 'Κωδικός KEN',
    hc.insurance AS 'Ασφαλιστικός Φορέας',
    COUNT(*) AS 'Πλήθος Νοσηλειών',
    SUM(hc.base_ken_cost) AS 'Συνολικό Βασικό Κόστος KEN',
    SUM(hc.extra_ken_cost) AS 'Πρόσθετη Χρέωση (Υπέρβαση ΜΔΝ)',
    SUM(IFNULL(pc.total_proc_cost, 0)) AS 'Έσοδα Χειρουργείων',
    SUM(IFNULL(lc.total_lab_cost, 0)) AS 'Έσοδα Εργαστηριακών',
    SUM(hc.total_hosp_cost + IFNULL(pc.total_proc_cost, 0) +
    IFNULL(lc.total_lab_cost, 0)) AS 'Συνολικά Έσοδα'
FROM hosp_costs hc
LEFT JOIN proc_costs pc ON hc.patient_amka = pc.patient_amka AND
hc.entry_date = pc.entry_date
LEFT JOIN lab_costs lc ON hc.patient_amka = lc.patient_amka AND
hc.entry_date = lc.entry_date
GROUP BY
    hc.dept_name,
    hc.eval_year,
    hc.ken_code,
    hc.insurance
ORDER BY
    hc.dept_name ASC,
    hc.eval_year DESC,
    'Συνολικά Έσοδα' DESC;

```


Αρχικά φτιάξαμε 3 virtual tables τα οποία οργανώνουν το καθένα όλα τα έσοδα από όλες τις νοσηλείες, όλες τις ιατρικές εξετάσεις και όλες τις επεμβάσεις. Το hosp_costs μαζεύει όλες τις νοσηλείες και κρατάει τα κόστωι των κεν της κάθε νοσηλείας μαζί και με τα έξτρα κόστωι διαμονής. Το proc_cost προσθέτει όλα τα κόστη από τις επεμβάσεις που έκανε κάθε ασθενής ανά μία επίσκεψη (γιατί μπορεί να έκανε και πάνω από μια επέμβαση στη διαμονή του) και την ίδια διαδικασία κάνει και το lab_costs για τις εξετάσεις. Τέλος συνδυάζουμε όλα τα συνολικά κόστωι που πηγάζουν από αυτούς τους πίνακες και πήραμε τα συνολικά έσοδα ανά τμήμα, έτος, κωδικό ΚΕΝ και ασφαλιστικό φορέα μαζί με το πλήθος νοσηλειών που έχει ο καθένας από αυτούς τους συνδυασμούς.

Q2: Για συγκεκριμένη ειδικότητα ιατρού, βρείτε όλους τους ιατρούς που ανήκουν σε αυτήν, με ένδειξη αν είχαν εφημερία το τρέχον έτος και πόσες επεμβάσεις εκτέλεσαν ως κύριοι χειρουργοί.

```
SELECT
    s.amka,
    s.first_name,
    s.last_name,
    d.speciality,
    d.doc_rank,
    CASE
        WHEN EXISTS (
            SELECT 1
            FROM shift_staff ss
            WHERE ss.staff_amka = d.amka
                  AND YEAR(ss.date) = YEAR(CURDATE())
        ) THEN 'Ναι'
        ELSE 'Όχι'
    END AS eixe_efimeria_fetos,
    COUNT(pr.procedure_id) AS epemvaseis_os_kyrios_xeirourgos
FROM doctor d
JOIN staff s ON s.amka = d.amka
LEFT JOIN `procedure` pr ON pr.main_surgeon_amka = d.amka
AND YEAR(pr.entry_date) = YEAR(CURDATE())
WHERE d.speciality = 'ΚΑΡΔΙΟΛΟΓΙΑ'
GROUP BY
    s.amka;
```

Σε αυτό το query αρχικά παίρνουμε όλους τους ιατρούς από μία ειδικότητα (έστω εδώ καρδιολογία) και με το LEFT JOIN παίρνουμε και όλα τα στοιχεία από τις επεμβάσεις χωρίς να σβήνονται αυτοί που δεν έχουν κάνει ακόμα επέμβαση. Έτσι και με τα στοιχεία από το staff παίρνουμε τα στοιχεία των ιατρών και ελέγχουμε με το IF statement αν είχε εφημερία εκείνο το έτος και μετράμε πόσες επεμβάσεις έχει κάνει αυτό το έτος.

Q3: Βρείτε ποιοι ασθενείς έχουν νοσηλευτεί περισσότερες από 3 φορές στο ίδιο τμήμα, με το συνολικό κόστος νοσηλείας τους.

```
SELECT
    p.amka,
    p.first_name AS 'Όνομα',
    p.last_name AS 'Επώνυμο',
    h.dept_name AS 'Τμήμα',
    COUNT(*) AS 'Πλήθος Νοσηλειών',
    SUM(h.cost) AS 'Συνολικό Κόστος'
FROM hospitalization h
JOIN patient p ON h.patient_amka = p.amka
GROUP BY p.amka, h.dept_name
HAVING COUNT(*) > 3
ORDER BY 'Συνολικό Κόστος' DESC;
```

Στο συγκεκριμένο ερώτημα, συνδυάζουμε αρχικά τα στοιχεία των νοσηλειών (hospitalization) με τα προσωπικά στοιχεία των ασθενών (patient) μέσω του κοινού τους πεδίου (AMKA). Στη συνέχεια, ομαδοποιούμε τα δεδομένα ανά ασθενή και ανά νοσοκομειακό τμήμα, ώστε να μπορούμε να μετρήσουμε πόσες φορές έχει εισαχθεί ο ίδιος άνθρωπος στο ίδιο τμήμα, αλλά και να προσθέσουμε το κόστος αυτών των νοσηλειών. Τέλος, χρησιμοποιώντας το κριτήριο HAVING COUNT(*) > 3, απομονώνουμε και εμφανίζουμε μόνο εκείνους τους ασθενείς που παρουσιάζουν υψηλή συχνότητα επανεισαγωγών (πάνω από 3 φορές), ταξινομώντας τους από το μεγαλύτερο συνολικό κόστος προς το μικρότερο.

Q4: Για συγκεκριμένο ιατρό, βρείτε τον μέσο όρο αξιολογήσεων των ασθενών του (κριτήριο: Ποιότητα ιατρικής φροντίδας) και τη Συνολική εντύπωση νοσηλείας.

```
SELECT
    ed.doctor_amka AS 'AMKA γιατρού',
    s.first_name AS 'Όνομα',
    s.last_name AS 'Επώνυμο',
    AVG(ed.medical_care) AS 'Μέσος όρος φροντίδας',
    AVG(eh.overall_experience) AS 'Συνολική εντύπωση'
FROM evaluation_doctor ed
JOIN evaluation_hospital eh
    ON ed.patient_amka = eh.patient_amka
    AND ed.entry_date = eh.entry_date
JOIN staff s ON ed.doctor_amka = s.amka
WHERE doctor_amka = '12127283498'
GROUP BY ed.doctor_amka;
```

Το παραπάνω query επιστρέφει για τον συγκεκριμένο ιατρό το AMKA του, το ονοματεπώνυμό του, τον μέσο όρο αξιολόγησης της ποιότητας ιατρικής φροντίδας από τους ασθενείς που τον αξιολόγησαν, καθώς και τον μέσο όρο της συνολικής εντύπωσης της νοσηλείας τους. Συγκεκριμένα, γίνεται JOIN μεταξύ των πινάκων **evaluation_doctor** και **evaluation_hospital** βάσει των πεδίων **patient_amka** και **entry_date**, ώστε να συνδυαστούν οι αξιολογήσεις ιατρού και νοσηλείας που αφορούν την ίδια νοσηλεία. Στη συνέχεια γίνεται JOIN με τον πίνακα **staff** για να ανακτηθεί το ονοματεπώνυμο του ιατρού. Το φίλτρο *WHERE doctor_amka = '12127283498'* περιορίζει τα αποτελέσματα στον συγκεκριμένο ιατρό, και το *GROUP BY ed.doctor_amka* εξασφαλίζει μία γραμμή αποτελέσματος με τους συγκεντρωτικούς μέσους όρους.

Αρχικά τροποποιούμε το παραπάνω μας κώδικα με τη προσθήκη της εντολής EXPLAIN, με την οποία λαμβάνουμε το πλάνο εκτέλεσης που θα ακολουθήσει η MySQL για το συγκεκριμένο SELECT. Αναλυτικότερα μας ενημερώνει όσον αφορά:

1. **id:** Αύξων αριθμός του επιμέρους μέρους του query. Αν έχουμε subqueries ή UNION, θα δούμε πολλαπλά ids.
2. **select_type:**
 - SIMPLE → απλό query χωρίς subqueries
 - PRIMARY → το κύριο query
 - SUBQUERY → υποερώτημα σε WHERE clause
 - DEPENDENT SUBQUERY → υποερώτημα που εξαρτάται από εξωτερικές τιμές
3. **table:** Ποιος πίνακας (στη προκειμένη περίπτωση alias) διαβάζεται.
4. **partitions:** Σε ποιο partition (αν υπάρχει) θα κοιτάξει.
5. **type:**

- ALL → full table scan, το οποίο προφανώς είναι μη επιθυμητό
 - index → scan μόνο του index
 - range → χρησιμοποίηση εύρους σε ευρετήριο
 - ref → αναζήτηση με index όταν υπάρχουν συγκρίσεις με ισότητα
 - eq_ref → ιδανικό για PK/FK joins
 - const/system → σταθερός χρόνος
6. **possible_keys**: Ποια indexes θα μπορούσαν να χρησιμοποιηθούν με βάση τα φιλτραρίσματά μας
 7. **key**: Ποιο index επέλεξε τελικά ο optimizer.
 8. **key_len**: Μήκος σε bytes του κομματιού του index που χρησιμοποιήθηκε.
 9. **ref**: Ποιο πεδίο συγκρίνεται με το index
 10. **rows**: Εκτίμηση αριθμού γραμμών που διαβάστηκαν για να βρεθεί το αποτέλεσμα
 11. **filtered**: Εκτίμηση ποσοστού γραμμών που περνούν τα φίλτρα ανάγνωσης
 12. **Extra**: Πρόσθετες πληροφορίες βελτιστοποίησης:
 - Using index → cover query (δε χρειάζεται ανάγνωση του table, μόνο του index)
 - Using where → εφαρμογή WHERE μετά από index/range scan
 - Using temporary → δημιουργία προσωρινού πίνακα (π.χ. για GROUP BY)
 - Using filesort → εξωτερική ταξινόμηση (ORDER BY χωρίς κατάλληλο index)

EXPLAIN

SELECT

```

ed.doctor_amka AS 'ΑΜΚΑ γιατρού',
s.first_name AS 'Όνομα',
s.last_name AS 'Επώνυμο',
AVG(ed.medical_care) AS 'Μέσος όρος φροντίδας',
AVG(eh.overall_experience) AS 'Συνολική εντύπωση'
FROM evaluation_doctor ed
JOIN evaluation_hospital eh
    ON ed.patient_amka = eh.patient_amka
    AND ed.entry_date = eh.entry_date
JOIN staff s ON ed.doctor_amka = s.amka
WHERE doctor_amka = '12127283498'
GROUP BY ed.doctor_amka;
```

	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
▶	1	SIMPLE	s	const	PRIMARY	PRIMARY	44	const	1	
	1	SIMPLE	ed	ref	PRIMARY, fk_eval_doc_doctor_idx_eval_doc_amka	fk_eval_doc_doctor	44	const	1	Using index condition
	1	SIMPLE	eh	eq_ref	PRIMARY, idx_eval_hosp_patient	PRIMARY	49	ygeiopolis.ed.patient_amka, ygeiopolis.ed.entry...	1	

Στο πλάνο εκτέλεσης βλέπουμε ότι το ερώτημα αποτελείται από τρία απλά (SIMPLE) βήματα. Η ερώτηση ξεκινάει από τον πίνακα *s* (*STAFF*) κι επειδή φιλτράρει με το πρωτεύον κλειδί (σταθερή τιμή), η MySQL τον διαβάζει σαν const. Έπειτα ενώνει τον πίνακα *ed* (*EVALUATION_DOCTOR*) χρησιμοποιώντας το index *fk_eval_doc_doctor*, οπότε το access type είναι ref και έτσι εντοπίζονται γρήγορα οι αξιολογήσεις του συγκεκριμένου ιατρού. Στο Extra εμφανίζεται "Using index condition", που σημαίνει ότι η MySQL εφαρμόζει επιπλέον φίλτρα απευθείας στο index, αποφεύγοντας περιττές αναγνώσεις από τον πίνακα. Τέλος, ενώνει τον πίνακα *eh* (*EVALUATION_HOSPITAL*) με access type eq_ref στο PRIMARY KEY, αφού οι τιμές σύγκρισης *patient_amka* και *entry_date* προέρχονται από τον πίνακα *ed* και όχι ως σταθερές τιμές. Οι εκτιμώμενες rows είναι 1 σε κάθε βήμα και ο optimizer επιλέγει αυτόματα τους βέλτιστους δείκτες σε κάθε βήμα — μια ένδειξη αποτελεσματικής χρήσης των ευρετηρίων.

EXPLAIN

SELECT

```

ed.doctor_amka AS 'AMKA γιατρού',
s.first_name AS 'Όνομα',
s.last_name AS 'Επώνυμο',
AVG(ed.medical_care) AS 'Μέσος όρος φροντίδας',
AVG(eh.overall_experience) AS 'Συνολική εντύπωση'
FROM evaluation_doctor ed FORCE INDEX (idx_eval_doc_amka)
JOIN evaluation_hospital eh
  ON ed.patient_amka = eh.patient_amka
  AND ed.entry_date = eh.entry_date
JOIN staff s ON ed.doctor_amka = s.amka
WHERE doctor_amka = '12127283498'
GROUP BY ed.doctor_amka;

```

	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
►	1	SIMPLE	s	const	PRIMARY	PRIMARY	44	const	1	
	1	SIMPLE	ed	ref	idx_eval_doc_amka	idx_eval_doc_amka	44	const	1	Using index condition
	1	SIMPLE	eh	eq_ref	PRIMARY,idx_eval_hosp_patient	PRIMARY	49	ygeiopolis.ed.patient_amka,ygeiopolis.ed.entry...	1	

Μετά το FORCE INDEX, η MySQL συνέχισε να χρησιμοποιεί το PRIMARY KEY ως INDEX για τον πίνακα *s* (STAFF) με τύπο αναζήτησης const, όμως για τον πίνακα *ed* (EVALUATION_DOCTOR) αναγκάστηκε να χρησιμοποιήσει το index **idx_eval_doc_amka** αντί του βέλτιστου `fk_eval_doc_doctor` που είχε επιλέξει ο optimizer στην προηγούμενη εκτέλεση. Αν και το access type παραμένει ref και οι εκτιμώμενες rows είναι 1, το index **idx_eval_doc_amka** είναι λιγότερο επιλεκτικό για το συγκεκριμένο JOIN σε σχέση με το `fk_eval_doc_doctor`, το οποίο ο optimizer θα επέλεγε φυσιολογικά. Για τον πίνακα *eh* (EVALUATION_HOSPITAL) δεν παρατηρείται αλλαγή.

Συνολικά, το FORCE INDEX δεν οδήγησε σε δραματική επιδείνωση όπως συμβαίνει σε άλλες περιπτώσεις, αλλά ο optimizer έχασε την ελευθερία να επιλέξει το βέλτιστο index για το JOIN στον *ed*, δείχνοντας ότι τα index hints πρέπει να εφαρμόζονται με προσοχή, ακόμα και όταν το αποτέλεσμα φαίνεται παρόμοιο, ο optimizer γνωρίζει καλύτερα ποιο index εξυπηρετεί αποδοτικά κάθε JOIN.

Πάμε τώρα να αναλύσουμε και αυτό το ερώτημα με τη προσθήκη των εντολών:

SET PROFILING = 1: Ενεργοποιεί τον εσωτερικό profiler της MySQL, ώστε κάθε επόμενη δήλωση να καταγράφεται με τον χρόνο εκτέλεσής της.

SHOW PROFILES: Μετά την εκτέλεση του query, εμφανίζει το χρόνο που χρειάστηκε για να τρέξουν σε δευτερόλεπτα.

Έτσι λοιπόν έχουμε τους εξής χρόνους πριν και μετά τη προσθήκη FORCE INDEX:

- Χωρίς FORCE INDEX:

```
1      0.00045290  explain select ed.doctor_amka as 'AMKA γιστρού', s.first_name as 'Όνομο', s.last_name as 'Επώνυμο'...
```

- Με FORCE INDEX:

```
2      0.00059600  explain select ed.doctor_amka as 'AMKA γιστρού', s.first_name as 'Όνομο', s.last_name as 'Επώνυμο'...
```

Παρατηρούμε πως ο χρόνος εκτέλεσης αυξάνεται με τη χρήση FORCE INDEX: από *0.00045290s* χωρίς FORCE INDEX σε *0.00059600s* με FORCE INDEX. Αυτό είναι αναμενόμενο αφού, όπως φάνηκε και από το EXPLAIN, η χρήση FORCE INDEX αναγκάζει τη MySQL να χρησιμοποιήσει το **idx_eval_doc_amka** αντί του βέλτιστου **fk_eval_doc_doctor** που θα επέλεγε αυτόματα ο optimizer. Συμπερασματικά, η MySQL επιλέγει από μόνη της τους πιο αποδοτικούς δείκτες για το συγκεκριμένο query, οπότε τα FORCE INDEX πρέπει να εφαρμόζονται μόνο όταν αποδεδειγμένα ο optimizer κάνει suboptimal επιλογές — στη συγκεκριμένη περίπτωση η παρέμβαση επιβάρυνε την απόδοση αντί να τη βελτιώσει.

Q5: Βρείτε τους νέους ιατρούς (ηλικία < 35 ετών) που έχουν εκτελέσει τις περισσότερες χειρουργικές επεμβάσεις ως κύριοι χειρουργοί.

```
WITH young_docs as (  
    SELECT  
        s.first_name AS 'Όνομα',  
        s.last_name AS 'Επώνυμο',  
        s.age AS 'Ηλικία',  
        COUNT(p.procedure_id) AS most_procs  
    FROM `procedure` p  
    JOIN procedures ps ON p.procedure_id = ps.procedure_id  
    JOIN staff s ON p.main_surgeon_amka = s.amka  
    JOIN doctor d ON s.amka = d.amka  
    WHERE s.age < 35  
    AND ps.type = 'Χειρουργική'  
    GROUP BY s.amka  
)  
SELECT * FROM young_docs  
WHERE most_procs = (SELECT MAX(most_procs) FROM young_docs);
```

Σε αυτό το ερώτημα, δημιουργούμε αρχικά έναν εικονικό πίνακα (young_docs) ο οποίος απομονώνει τους γιατρούς κάτω των 35 ετών, διασταυρώνοντας τα στοιχεία του προσωπικού (staff), των γιατρών (doctor), των ιατρικών πράξεων (procedure) και των τύπων τους (procedures). Μέσα σε αυτόν τον πίνακα, φιλτράρουμε τις επεμβάσεις ώστε να μετρήσουμε μόνο όσες είναι «Χειρουργικές» και στις οποίες ο γιατρός συμμετείχε ως κύριος χειρουργός, ομαδοποιώντας τα αποτελέσματα ανά AMKA για να βρούμε το πλήθος των χειρουργείων για κάθε νέο γιατρό. Τέλος, στο κύριο ερώτημα, χρησιμοποιούμε την υποσυνάρτηση SELECT MAX(most_procs) για να συγκρίνουμε τις επιδόσεις τους και να εμφανίσουμε αποκλειστικά εκείνον ή εκείνους τους νέους γιατρούς που έχουν εκτελέσει τον μέγιστο αριθμό χειρουργικών επεμβάσεων στο νοσοκομείο.

Q6: Για συγκεκριμένο ασθενή, βρείτε το ιστορικό νοσηλειών του, τις αντίστοιχες διαγνώσεις (ICD-10), το συνολικό κόστος ανά νοσηλεία και τον μέσο όρο αξιολόγησής του.

SELECT

```
h.entry_date AS 'Ημερομηνία εισαγωγής',  
h.exit_date AS 'Ημερομηνία εξόδου',  
DATEDIFF(h.exit_date, h.entry_date) AS 'Ημέρες νοσηλείας',  
h.dept_name AS 'Τμήμα',  
h.room_number AS 'Αριθμός δωματίου',  
h.entry_diagnosis AS 'ICD εισαγωγής',  
d_in.description AS 'Διάγνωση εισαγωγής',  
h.exit_diagnosis AS 'ICD εξόδου',  
d_out.description AS 'Διάγνωση εξόδου',  
h.cost AS 'Συνολικό κόστος',  
AVG(eh.overall_experience) AS 'Μέσος όρος αξιολόγησης'
```

FROM hospitalization h

JOIN diagnosis d_in

```
ON h.entry_diagnosis = d_in.ICD
```

LEFT JOIN diagnosis d_out

```
ON h.exit_diagnosis = d_out.ICD
```

LEFT JOIN evaluation_hospital eh

```
ON h.patient_amka = eh.patient_amka
```

```
AND h.entry_date = eh.entry_date
```

WHERE h.patient_amka = '26124400192'

GROUP BY h.entry_date;

Το παραπάνω query επιστρέφει για τον συγκεκριμένο ασθενή το ιστορικό νοσηλειών του, με τις ημερομηνίες εισαγωγής και εξόδου, τις διαγνώσεις εισαγωγής και εξόδου (κωδικοί ICD-10), το συνολικό κόστος κάθε νοσηλείας και τον μέσο όρο αξιολόγησής του. Συγκεκριμένα, γίνεται JOIN μεταξύ του πίνακα **hospitalization** και του πίνακα **diagnosis** δύο φορές — μία για τη διάγνωση εισαγωγής (alias d_in) και μία για τη διάγνωση εξόδου (alias d_out). Στη συνέχεια γίνεται JOIN με τον πίνακα **evaluation_hospital** για να ανακτηθεί η αξιολόγηση της νοσηλείας. Το φίλτρο **WHERE** περιορίζει τα αποτελέσματα στον συγκεκριμένο ασθενή.

Αρχικά τροποποιούμε το παραπάνω μας κώδικα με τη προσθήκη της εντολής EXPLAIN, με την οποία λαμβάνουμε το πλάνο εκτέλεσης που θα ακολουθήσει η MySQL για το συγκεκριμένο SELECT.

```
EXPLAIN
SELECT
    h.entry_date AS 'Ημερομηνία εισαγωγής',
    h.exit_date AS 'Ημερομηνία εξόδου',
    DATEDIFF(h.exit_date, h.entry_date) AS 'Ημέρες νοσηλείας',
    h.dept_name AS 'Τμήμα',
    h.room_number AS 'Αριθμός δωματίου',
    h.entry_diagnosis AS 'ICD εισαγωγής',
    d_in.description AS 'Διάγνωση εισαγωγής',
    h.exit_diagnosis AS 'ICD εξόδου',
    d_out.description AS 'Διάγνωση εξόδου',
    h.cost AS 'Συνολικό κόστος',
    AVG(eh.overall_experience) AS 'Μέσος όρος αξιολόγησης'
FROM hospitalization h
JOIN diagnosis d_in
    ON h.entry_diagnosis = d_in.ICD
LEFT JOIN diagnosis d_out
    ON h.exit_diagnosis = d_out.ICD
LEFT JOIN evaluation_hospital eh
    ON h.patient_amka = eh.patient_amka
    AND h.entry_date = eh.entry_date
WHERE h.patient_amka = '26124400192'
GROUP BY h.entry_date;
```

	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
▶	1	SIMPLE	h	ref	PRIMARY,fk_hosp_entry_diagnosis_idx,idx_hos...	PRIMARY	44	const	8	Using where
	1	SIMPLE	d_in	eq_ref	PRIMARY	PRIMARY	42	ygeiopolis.h.entry_diagnosis	1	
	1	SIMPLE	d_out	eq_ref	PRIMARY	PRIMARY	42	ygeiopolis.h.exit_diagnosis	1	Using where
	1	SIMPLE	eh	eq_ref	PRIMARY,idx_eval_hosp_patient	PRIMARY	49	ygeiopolis.h.patient_amka,ygeiopolis.h.entry_d...	1	

Στο πλάνο εκτέλεσης βλέπουμε ότι το ερώτημα αποτελείται από τέσσερα απλά (SIMPLE) βήματα. Η ερώτηση ξεκινάει από τον πίνακα *h* (*HOSPITALIZATION*) με access type ref στο PRIMARY KEY, καθώς φιλτράρει με σταθερή τιμή στο patient_amka, εντοπίζοντας έτσι γρήγορα όλες οι νοσηλείες του συγκεκριμένου ασθενή. Το Extra εμφανίζει "Using where", που σημαίνει ότι εφαρμόζονται επιπλέον φίλτρα μετά την ανάγνωση από το index, ενώ οι εκτιμώμενες rows είναι 8 (ο αριθμός νοσηλειών του ασθενή). Έπειτα ενώνει τον πίνακα *d_in* (*DIAGNOSIS για διάγνωση εισαγωγής*) με access type eq_ref στο PRIMARY KEY, αφού η τιμή entry_diagnosis προέρχεται από τον πίνακα *h*, με εκτιμώμενες rows = 1. Στη συνέχεια

ενώνει τον πίνακα *d_out* (DIAGNOSIS για διάγνωση εξόδου) με access type eq_ref στο PRIMARY KEY βάσει του exit_diagnosis, με εκτιμώμενες rows = 1 και Extra "Using where" για τον επιπλέον έλεγχο NULL τιμής. Τέλος, ενώνει τον πίνακα *eh* (EVALUATION_HOSPITAL) με access type eq_ref στο PRIMARY KEY, αφού οι τιμές σύγκρισης patient_amka και entry_date προέρχονται από τον πίνακα *h*. Οι εκτιμώμενες rows είναι 1, δεν χρειάζεται filesort ή προσωρινός πίνακας, και ο optimizer επιλέγει αυτόματα τους βέλτιστους δείκτες σε κάθε βήμα, ένδειξη αποτελεσματικής χρήσης των ευρετηρίων.

EXPLAIN

SELECT

```

h.entry_date AS 'Ημερομηνία εισαγωγής',
h.exit_date AS 'Ημερομηνία εξόδου',
DATEDIFF(h.exit_date, h.entry_date) AS 'Ημέρες νοσηλείας',
h.dept_name AS 'Τμήμα',
h.room_number AS 'Αριθμός δωματίου',
h.entry_diagnosis AS 'ICD εισαγωγής',
d_in.description AS 'Διάγνωση εισαγωγής',
h.exit_diagnosis AS 'ICD εξόδου',
d_out.description AS 'Διάγνωση εξόδου',
h.cost AS 'Συνολικό κόστος',
AVG(eh.overall_experience) AS 'Μέσος όρος αξιολόγησης'
FROM hospitalization h FORCE INDEX (idx_hosp_patient)
JOIN diagnosis d_in
  ON h.entry_diagnosis = d_in.ICD
LEFT JOIN diagnosis d_out
  ON h.exit_diagnosis = d_out.ICD
LEFT JOIN evaluation_hospital eh FORCE INDEX (idx_eval_hosp_patient)
  ON h.patient_amka = eh.patient_amka
  AND h.entry_date = eh.entry_date
WHERE h.patient_amka = '26124400192'
GROUP BY h.entry_date;
```

	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
►	1	SIMPLE	h	ref	idx_hosp_patient	idx_hosp_patient	44	const	8	Using index condition; Using where
	1	SIMPLE	d_in	eq_ref	PRIMARY	PRIMARY	42	ygeiopolis.h.entry_diagnosis	1	
	1	SIMPLE	d_out	eq_ref	PRIMARY	PRIMARY	42	ygeiopolis.h.exit_diagnosis	1	Using where
	1	SIMPLE	eh	ref	idx_eval_hosp_patient	idx_eval_hosp_patient	49	ygeiopolis.h.patient_amka,ygeiopolis.h.entry_d...	1	

Μετά το FORCE INDEX, η MySQL αναγκάστηκε να χρησιμοποιήσει το **idx_hosp_patient** αντί του βέλτιστου PRIMARY KEY για τον πίνακα *h*. Αν και ο αριθμός εκτιμώμενων rows παραμένει 8 και το access type είναι πάλι ref, το Extra αλλάζει σε **Using index condition; Using where**, που σημαίνει ότι η MySQL πρέπει να εφαρμόσει επιπλέον φίλτρα εντός του index αλλά και να επιστρέψει στον πίνακα για τα υπόλοιπα πεδία (αύξηση I/O σε σχέση με το PRIMARY KEY scan). Για τον πίνακα *eh*, η χρήση

του **idx_eval_hosp_patient** υποβαθμίζει το access type από eq_ref σε ref, δηλαδή αντί για εγγυημένα μοναδική αντιστοίχιση, η MySQL εκτελεί πιο γενική αναζήτηση στο index. Οι πίνακες d_in και d_out δεν επηρεάζονται.

Έχουμε τους εξής χρόνους πριν και μετά τη προσθήκη FORCE INDEX:

- Χωρίς FORCE INDEX:

```
| 16          0.00053330  explain select h.entry_date as 'Ημερομηνία εισαγωγής', h....
```

- Με FORCE INDEX:

```
| 17          0.00047100  explain select h.entry_date as 'Ημερομηνία εισαγωγής', h....
```

Παρατηρούμε πως ο χρόνος εκτέλεσης μειώνεται με τη χρήση FORCE INDEX: από 0.00053330s χωρίς FORCE INDEX σε 0.00047100s με FORCE INDEX. Ωστόσο, αυτό δεν σημαίνει απαραίτητα ότι το FORCE INDEX είναι η βέλτιστη επιλογή, όπως φάνηκε και από το EXPLAIN, η χρήση του αναγκάζει τη MySQL να εγκαταλείψει το PRIMARY KEY για τον πίνακα h και να υποβαθμίσει το JOIN του eh από eq_ref σε ref. Η μικρή χρονική βελτίωση που παρατηρείται, μπορεί να οφείλεται στο συγκεκριμένο μέγεθος και κατανομή των δεδομένων στο dataset μας, και δεν εγγυάται αντίστοιχο αποτέλεσμα σε περιβάλλον με μεγαλύτερο όγκο δεδομένων. Συμπερασματικά, όπως αναφέρθηκε και στο Q4, τα FORCE INDEX πρέπει να εφαρμόζονται με προσοχή και μόνο ύστερα από εκτεταμένες μετρήσεις.

Q7:Βρείτε για κάθε δραστική ουσία τον αριθμό ασθενών που έχουν δηλώσει αλλεργία και τον αριθμό φαρμάκων που την περιέχουν, ταξινομημένα κατά συνολικό αριθμό αλλεργικών ασθενών.

```
SELECT
    a.substance_id,
    s.name AS drastiki_ousia,
    COUNT(DISTINCT a.patient_amka) AS arithmos_allergikon_asthenwn,
    COUNT(DISTINCT ds.drug_id) AS arithmos_farmakon
FROM active_substance s
LEFT JOIN allergy a ON a.substance_id = s.substance_id
LEFT JOIN drug_substance ds ON ds.substance_id = s.substance_id
GROUP BY s.substance_id
ORDER BY arithmos_allergikon_asthenwn DESC;
```

Ξεκινήσαμε με βασικό κορμό τον πίνακα και τον ενώσαμε με LEFT JOIN με τους πίνακες των αλλεργιών και των φαρμάκων το μπήκε για να μην κοπεί καμία απολύτως ουσία από το αποτέλεσμα, ακόμα και αν δεν έχει προκαλέσει ποτέ αλλεργία. Για να βγουν σωστά τα νούμερα χρησιμοποιήσαμε DISTINCT, ώστε να μετρήσουμε αυστηρά

μοναδικούς ασθενείς και μοναδικά φάρμακα, αποφεύγοντας διπλοεγγραφές στο ιστορικό. Τέλος, ομαδοποιήσαμε τα δεδομένα ανά ουσία και τα ταξινομήσαμε φθίνουσα, φέρνοντας έτσι τις πιο "επικίνδυνες" ουσίες με τα περισσότερα περιστατικά στην κορυφή της λίστας.

Q8: Βρείτε το προσωπικό (ιατροί, νοσηλευτές, διοικητικό προσωπικό) που δεν έχει προγραμματισμένη εφημερία σε συγκεκριμένη ημερομηνία και τμήμα.

```
set @dept = 'ΠΑΘΟΛΟΓΙΑ';  
set @date = '2025-05-05';
```

```
WITH dept_staff AS(  
    SELECT amka, dept_name FROM nurse  
    UNION  
    SELECT amka, dept_name FROM personel  
    UNION  
    SELECT doc_amka AS amka, dept_name FROM department_doctor  
)  
SELECT  
    s.amka,  
    s.last_name AS 'Επίθετο',  
    s.first_name AS 'Όνομα',  
    s.staff_type AS 'Ειδικότητα'  
FROM staff s  
JOIN dept_staff ds ON s.amka = ds.amka AND ds.dept_name = @dept  
LEFT JOIN shift_staff ss ON s.amka = ss.staff_amka AND ss.date = @date  
WHERE ss.staff_amka IS NULL  
ORDER BY s.staff_type, s.last_name;
```

Σε αυτό το ερώτημα, δημιουργούμε αρχικά έναν εικονικό πίνακα (dept_staff) ο οποίος ενώνει τη διασπορά του προσωπικού από τρεις διαφορετικούς πίνακες (nurse, personel και department_doctor), συγκεντρώνοντας σε μία ενιαία λίστα το AMKA και το τμήμα για κάθε νοσηλεύτη, διοικητικό υπάλληλο και γιατρό. Στη συνέχεια, συνδέουμε τη λίστα αυτή με τα πλήρη στοιχεία του προσωπικού (staff), φιλτράροντας τα αποτελέσματα ώστε να κρατήσουμε μόνο όσους ανήκουν οργανικά στο τμήμα που μας ενδιαφέρει (π.χ. 'ΠΑΘΟΛΟΓΙΑ'). Τέλος, πραγματοποιούμε μια αριστερή εξωτερική σύνδεση (LEFT JOIN) με τον πίνακα των βαρδιών (shift_staff) για τη συγκεκριμένη ημερομηνία και, μέσω του κριτηρίου WHERE ss.staff_amka IS NULL, απομονώνουμε και εμφανίζουμε τα ονόματα εκείνων των εργαζομένων που δεν βρέθηκε καμία καταχωρημένη βάρδια για αυτούς τη δεδομένη ημέρα στο συγκεκριμένο τμήμα.

Q9: Βρείτε ποιοι ασθενείς νοσηλεύτηκαν τον ίδιο αριθμό ημερών σε διάστημα ενός έτους, με συνολική διάρκεια άνω των 15 ημερών.

```
WITH yearly_patients AS (  
    SELECT  
        h.patient_amka,  
        p.first_name,  
        p.last_name,  
        year(h.entry_date) AS hosp_year,  
        sum(datediff(h.exit_date, h.entry_date)) AS total_days  
    FROM hospitalization h  
    JOIN patient p ON h.patient_amka = p.amka  
    WHERE h.exit_date IS NOT NULL  
    GROUP BY h.patient_amka, hosp_year  
    HAVING total_days > 15  
,  
shared AS (  
    SELECT  
        hosp_year AS hosp_year,  
        total_days  
    FROM yearly_patients  
    GROUP BY hosp_year, total_days  
    HAVING count(patient_amka) > 1  
)  
SELECT  
    yp.patient_amka,  
    yp.last_name AS 'Επίθετο',  
    yp.first_name AS 'Όνομα',  
    yp.hosp_year AS 'Έτος',  
    yp.total_days AS 'Διάρκεια Παραμονής'  
FROM yearly_patients yp  
JOIN shared sh ON yp.hosp_year = sh.hosp_year AND yp.total_days =  
sh.total_days  
ORDER BY yp.hosp_year desc, yp.total_days desc, yp.last_name;
```

Αρχικά φτιάξαμε το virtual table yearly_patients όπου με τη συνάρτηση SUM(DATEDIFF(...)) αθροίσαμε τις ημέρες νοσηλείας κάθε ασθενούς ανά έτος, κρατώντας με την HAVING total_days > 15 μόνο όσους ξεπέρασαν τις 15 μέρες. Μετά, σε ένα δεύτερο virtual table (shared), κάναμε GROUP BY στο έτος και τις ημέρες, χρησιμοποιώντας την HAVING COUNT(patient_amka) > 1 για να βρούμε ποιες ακριβώς διάρκειες είναι κοινές σε πάνω από έναν ασθενή. Τέλος, κάναμε JOIN αυτούς τους δύο πίνακες για να φιλτράρουμε την αρχική λίστα και να τυπώσουμε αυστηρά μόνο τους ασθενείς που έχουν την ίδια ακριβώς διάρκεια νοσηλείας το ίδιο έτος.

Q10: Πολλοί ασθενείς λαμβάνουν συνδυασμούς φαρμάκων. Βρείτε τα top-3 ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν ταυτόχρονα στον ίδιο ασθενή κατά την ίδια νοσηλεία, ταξινομημένα κατά συχνότητα εμφάνισης.

```
WITH hosp_drugs AS(
    SELECT
        pr.patient_amka AS patient,
        pr.hosp_entry_date AS entdate,
        ds.substance_id AS sub
    FROM prescription pr
    JOIN drug_substance ds ON pr.drug_id = ds.drug_id
)
SELECT
    hd1.sub AS 'Ουσία_1',
    hd2.sub AS 'Ουσία_2',
    count(*) AS 'Συχνότητα Εμφάνισης'
FROM hosp_drugs hd1
JOIN hosp_drugs hd2 ON hd1.patient = hd2.patient AND hd1.entdate =
hd2.entdate AND hd1.sub < hd2.sub
GROUP BY hd1.sub, hd2.sub
ORDER BY `Συχνότητα Εμφάνισης` desc
LIMIT 3;
```

Αρχικά δημιουργήσαμε το virtual table hosp_drugs για να αντιστοιχίσουμε τα ζευγάρια κάθε νοσηλείας (ασθενής και ημερομηνία) με τις αντίστοιχες δραστικές ουσίες τους μέσω της JOIN. Στη συνέχεια, εφαρμόσαμε ένα "Self Join" (ενώσαμε το virtual table με τον εαυτό του) για να βρούμε ποιες ουσίες συνυπάρχουν στην ίδια νοσηλεία. Εδώ χρησιμοποιήσαμε την συνθήκη `hd1.sub < hd2.sub`, η οποία εξασφαλίζει ότι θα δημιουργηθούν αυστηρά μοναδικά ζευγάρια (αποτρέποντας διπλοτυπίες του τύπου A-B και B-A ή το ταίριασμα μιας ουσίας με τον εαυτό της). Τέλος, χρησιμοποιήσαμε GROUP BY στα ζεύγη ουσιών για να μετρήσουμε με την COUNT(*) τη συχνότητα εμφάνισής τους, ταξινομήσαμε φθίνουσα, και με την εντολή LIMIT 3 κρατήσαμε αποκλειστικά τα τρία πιο συχνά.

Q11: Βρείτε όλους τους ιατρούς που έχουν εκτελέσει τουλάχιστον 5 λιγότερες επεμβάσεις από τον ιατρό με τις περισσότερες επεμβάσεις στο τρέχον έτος.

```
WITH curyear_surgeons AS (  
    SELECT  
        d.amka,  
        count(*) as total_procs  
    FROM doctor d  
    LEFT JOIN `procedure` pr ON d.amka = pr.main_surgeon_amka and  
year(entry_date) = year(curdate())  
    GROUP BY d.amka  
    ORDER BY total_procs desc  
) ,  
max_procs AS (  
    select max(total_procs) AS max_tots  
    from curyear_surgeons  
)  
SELECT  
    cs.amka,  
    s.last_name AS 'Επίθετο',  
    s.first_name AS 'Όνομα',  
    cs.total_procs AS 'Φετινές Επεμβάσεις'  
FROM curyear_surgeons cs  
JOIN staff s ON s.amka = cs.amka  
CROSS JOIN max_procs mp  
WHERE cs.total_procs <= mp.max_tots - 5  
ORDER BY cs.total_procs desc;
```

Στο ερώτημα αυτό, δημιουργούμε αρχικά δύο εικονικούς πίνακες για να οργανώσουμε τα στατιστικά των χειρουργείων. Ο πρώτος (curyear_surgeons) συγκεντρώνει όλους τους γιατρούς και καταγράφει το πλήθος των επεμβάσεων που έχει εκτελέσει ο καθένας ως κύριος χειρουργός μέσα στο τρέχον έτος, συνδέοντας τους πίνακες doctor και procedure. Ο δεύτερος πίνακας (max_procs) απομονώνει και κρατάει αποκλειστικά τον μέγιστο αριθμό επεμβάσεων που πέτυχε ο πρώτος σε επιδόσεις γιατρός για τη χρονιά αυτή. Τέλος, στο κύριο ερώτημα, ενώνουμε τις φετινές επιδόσεις των γιατρών με τα προσωπικά τους στοιχεία από τον πίνακα staff και, με τη βοήθεια μιας καθολικής σύγκρισης (CROSS JOIN), φιλτράρουμε τα αποτελέσματα ώστε να εμφανίσουμε μόνο όσους γιατρούς έχουν πραγματοποιήσει τουλάχιστον 5 λιγότερες επεμβάσεις από την κορυφαία φετινή επίδοση.

Q12: Βρείτε τον απαιτούμενο αριθμό προσωπικού ανά τμήμα και ανά βάρδια για συγκεκριμένη εβδομάδα, με ανάλυση ανά υποκλάση προσωπικού (ιατροί ανά ειδικότητα, νοσηλευτές ανά βαθμίδα, διοικητικό προσωπικό ανά ρόλο).

```
SELECT
    ss.dept_name AS 'Τμήμα',
    ss.`type` AS 'Βάρδια',
    CONCAT(MIN(ss.`date`), ' έως ', MAX(ss.`date`)) AS 'Εβδομάδα',
    s.staff_type AS 'Κλάδος Προσωπικού',
    COALESCE(d.speciality, n.`rank`, p.`role`, 'Άγνωστο') AS 'Υποκλάδος',
    COUNT(ss.staff_amka) AS 'Απαιτούμενο Προσωπικό'
FROM shift_staff ss
JOIN staff s ON ss.staff_amka = s.amka
LEFT JOIN doctor d ON s.amka = d.amka
LEFT JOIN nurse n ON s.amka = n.amka
LEFT JOIN personel p ON s.amka = p.amka
WHERE ss.`date` BETWEEN '2026-05-11' AND '2026-05-17'
GROUP BY
    ss.dept_name,
    ss.`type`,
    s.staff_type,
    COALESCE(d.speciality, n.`rank`, p.`role`, 'Άγνωστο')
ORDER BY
    ss.dept_name ASC,
    ss.`type` ASC,
    COUNT(ss.staff_amka) DESC;
```

Αρχικά πήραμε ως βάση τον πίνακα shift_staff και τον ενώσαμε με JOIN με τον πίνακα (staff). Για να πετύχουμε την ανάλυση ανά υποκλάση, συνδέσαμε τον πίνακα προσωπικού με LEFT JOIN προς τους τρεις εξειδικευμένους πίνακες (doctor, nurse και personel). Στο SELECT χρησιμοποιήσαμε την συνάρτηση COALESCE, η οποία "κοιτάζει" διαδοχικά τις στήλες της ειδικότητας, της βαθμίδας και του ρόλου, κρατώντας κάθε φορά αυτή που δεν είναι κενή, ώστε να τις συγκεντρώσει όλες σε μία στήλη ("Υποκλάδος"). Στο WHERE φιλτράραμε τις ημερομηνίες με την BETWEEN για τη συγκεκριμένη εβδομάδα που μας ζητήθηκε, ενώ με τη συνάρτηση CONCAT μαζί με τα MIN και MAX εμφανίζουμε το εύρος των ημερών. Τέλος, ομαδοποιήσαμε τα δεδομένα με GROUP BY ανά τμήμα, βάρδια και υποκλάδο, ώστε η COUNT(*) να μας δώσει τον ακριβή αριθμό εργαζομένων που απασχολούνται σε κάθε συνδυασμό.

Q13: Βρείτε για κάθε ιατρό όλη την ιεραρχία εποπτείας του, από τον άμεσο επόπτη έως τον Διευθυντή, με ένδειξη του επιπέδου σε κάθε βαθμίδα.

```
WITH RECURSIVE ierarxia AS (  
  SELECT  
    d.amka AS amka_iatrou,  
    s.first_name AS onoma_iatrou,  
    s.last_name AS epwnymo_iatrou,  
    d.doc_rank AS bathmida_iatrou,  
    d.supervisor_amka AS amka_epopti,  
    d.amka AS arxikos_iatros,  
    0 AS epipedo  
  FROM doctor d  
  JOIN staff s ON d.amka = s.amka  
  WHERE d.supervisor_amka IS NOT NULL  
  
  UNION ALL  
  
  SELECT  
    d2.amka AS amka_iatrou,  
    s2.first_name AS onoma_iatrou,  
    s2.last_name AS epwnymo_iatrou,  
    d2.doc_rank AS bathmida_iatrou,  
    d2.supervisor_amka AS amka_epopti,  
    ier.arxikos_iatros AS arxikos_iatros,  
    ier.epipedo+1 AS epipedo  
  FROM doctor d2  
  JOIN staff s2 ON d2.amka = s2.amka  
  JOIN ierarxia ier ON d2.amka = ier.amka_epopti  
  WHERE ier.amka_epopti IS NOT NULL  
)  
SELECT  
  s1.last_name AS 'Επώνυμο (αρχικού)',  
  s1.first_name AS 'Όνομα',  
  d1.doc_rank AS 'Βαθμίδα',  
  ier.epipedo AS 'Επίπεδο Εποπτείας',  
  ier.onoma_iatrou AS 'Όνομα (επόπτη)',  
  ier.epwnymo_iatrou AS 'Επώνυμο',  
  ier.bathmida_iatrou AS 'Βαθμίδα'  
FROM ierarxia ier  
JOIN doctor d1 ON ier.arxikos_iatros = d1.amka  
JOIN staff s1 ON ier.arxikos_iatros = s1.amka
```

```
WHERE ier.epipedo > 0
ORDER BY s1.last_name, ier.epipedo;
```

Για να αποτυπώσουμε την πλήρη αλυσίδα εποπτείας, χρησιμοποιήσαμε ένα αναδρομικό ερώτημα (WITH RECURSIVE) ώστε να "σκαρφαλώσουμε" στο δέντρο της ιεραρχίας. Στο αρχικό σκέλος (βάση της αναδρομής), επιλέξαμε όλους τους ιατρούς που έχουν επόπτη, θέτοντας το αρχικό επίπεδο στο 0 και "κλειδώνοντας" το AMKA τους (ως `arxikos_iatros`) για να θυμόμαστε από ποιον ξεκινήσαμε. Στο δεύτερο σκέλος, το ερώτημα λειτουργεί κυκλικά: ενώνει διαρκώς τον πίνακα των ιατρών με το αποτέλεσμα του προηγούμενου βήματος (ψάχνοντας τον επόπτη του επόπτη), αυξάνοντας κάθε φορά το επίπεδο (`epipedo + 1`) μέχρι να φτάσει στην κορυφή (π.χ. Διευθυντής). Τέλος, στο κεντρικό SELECT, ενώσαμε αυτή την παραγόμενη αλυσίδα ξανά με το προσωπικό για να εμφανίσουμε τα ονόματα, και φιλτράραμε με `epipedo > 0` ώστε να κρύψουμε το μηδενικό βήμα και να τυπώσουμε αποκλειστικά τους ανωτέρους του κάθε ιατρού.

Q14: Βρείτε ποιες κατηγορίες ICD-10 διαγνώσεων είχαν τον ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη, με τουλάχιστον 5 περιστατικά ανά έτος.

```
WITH diagnosis_counts AS (
    SELECT
        exit_diagnosis AS 'icd10',
        YEAR(entry_date) AS 'year_num',
        COUNT(*) AS 'incident_count'
    FROM hospitalization
    WHERE exit_diagnosis IS NOT NULL
    GROUP BY exit_diagnosis, YEAR(entry_date)
    HAVING COUNT(*) >= 5
)
```

```
SELECT
    dc1.icd10 AS 'Κωδικός ICD-10',
    dc1.year_num AS 'Έτος A',
    dc2.year_num AS 'Έτος B (Συνεχόμενο)',
    dc1.incident_count AS 'Πλήθος Περιστατικών'
FROM diagnosis_counts dc1
JOIN diagnosis_counts dc2 ON dc1.icd10 = dc2.icd10
    AND dc2.year_num = dc1.year_num + 1
WHERE dc1.incident_count = dc2.incident_count;
```

Στο ερώτημα αυτό, δημιουργούμε αρχικά έναν εικονικό πίνακα (diagnosis_counts) ο οποίος καταγράφει τη συχνότητα των ασθενειών ανά έτος. Συγκεκριμένα, αντλεί τις διαγνώσεις εξιτηρίου από τον πίνακα των νοσηλειών (hospitalization), τις ομαδοποιεί ανά κωδικό ICD-10 και ανά έτος εισαγωγής, και κρατάει μόνο εκείνους τους συνδυασμούς που εμφανίζουν τουλάχιστον 5 περιστατικά. Στη συνέχεια, στο κύριο ερώτημα, συνδέουμε τον πίνακα αυτόν με τον εαυτό του, θέτοντας δύο πολύ συγκεκριμένα κριτήρια: πρώτον, να αφορά την ίδια ακριβώς διάγνωση σε δύο συνεχόμενες χρονιές και, δεύτερον, ο αριθμός των περιστατικών αυτών των δύο ετών να είναι ακριβώς ο ίδιος. Έτσι, απομονώνουμε και εμφανίζουμε τις κατηγορίες διαγνώσεων που παρουσίασαν απόλυτη σταθερότητα σε συχνότητα από τη μία χρονιά στην επόμενη.

Q15: Βρείτε την κατανομή των περιστατικών triage ανά επίπεδο επείγοντος, με μέσο χρόνο αναμονής ανά επίπεδο, ποσοστό περιστατικών που οδήγησαν σε νοσηλεία και κατανομή παραπομπών ανά τμήμα.

```
WITH triage_data AS (  
    SELECT  
        t.urgency_level,  
        t.patient_amka,  
        t.arrival_time,  
        t.outcome,  
        h.dept_name,  
        TIMESTAMPDIFF(MINUTE, t.arrival_time, h.entry_date) AS duration  
    FROM triage t  
    LEFT JOIN hospitalization h  
        ON t.patient_amka = h.patient_amka  
        AND DATE(t.arrival_time) = DATE(h.entry_date)  
) ,
```

```
per_urgency AS (  
    SELECT  
        urgency_level,  
        COUNT(*) AS total_cases,  
        SUM(CASE WHEN outcome = 'ΝΟΣΗΛΕΙΑ' then 1 else 0 END) AS  
hosp_cases,  
        AVG(duration) AS avg_dur  
    FROM triage_data  
    GROUP BY urgency_level  
) ,
```

```

per_dept AS (
    SELECT
        urgency_level,
        dept_name,
        count(*) AS dept_cases
    FROM triage_data
    WHERE dept_name IS NOT NULL
    GROUP BY urgency_level, dept_name
),
per_dept_string AS(
    SELECT
        urgency_level,
        GROUP_CONCAT(CONCAT(dept_name, ' (', dept_cases, ') ') SEPARATOR ',
' ) AS dept_string
    FROM per_dept
    GROUP BY urgency_level
)
SELECT
    pu.urgency_level AS 'Επίπεδο Επείγοντος (1-5)',
    pu.total_cases AS 'Σύνολο Περιστατικών',
    ROUND(pu.avg_dur, 0) AS 'Μέσος Χρόνος Αναμονής (Λεπτά)',
    ROUND((pu.hosp_cases / pu.total_cases) * 100, 1) AS 'Ποσοστό Νοσηλείας (%)',
    IFNULL(pds.dept_string, 'Καμία Παραπομπή') AS 'Κατανομή Παραπομπών ανά Τμήμα'
FROM per_urgency pu
LEFT JOIN per_dept_string pds ON pu.urgency_level = pds.urgency_level
ORDER BY pu.urgency_level asc;

```

Αρχικά δημιουργήσαμε το virtual table triage_data, ενώνοντας τον πίνακα διαλογής με τις νοσηλείες για να υπολογίσουμε τον χρόνο αναμονής σε λεπτά μέσω της TIMESTAMPTDIFF. Έπειτα, στο per_urgency, ομαδοποιήσαμε τα δεδομένα ανά επίπεδο επείγοντος, χρησιμοποιώντας τη δομή SUM(CASE WHEN...) για να απομονώσουμε και να μετρήσουμε πόσα περιστατικά κατέληξαν σε νοσηλεία. Παράλληλα, μέσα από τα επόμενα WITH, μετρήσαμε τις παραπομπές ανά τμήμα και τις «συμπιέσαμε» σε ένα ενιαίο, ευανάγνωστο κείμενο χρησιμοποιώντας την GROUP_CONCAT. Στο τελικό SELECT, συνδυάσαμε όλα αυτά τα κομμάτια, υπολογίσαμε το ποσοστό νοσηλείας κάνοντας στρογγυλοποίηση με τη συνάρτηση ROUND, και εμφανίσαμε την πλήρη εικόνα ταξινομημένη ιεραρχικά από το επίπεδο 1 έως το 5.

Εφαρμογή

Το γραφικό περιβάλλον της εφαρμογής μας χτίστηκε με Python και Flask. Στόχος μας ήταν να φτιάξουμε ένα απλό και γρήγορο σύστημα που συνδέεται με τη MariaDB και διαχειρίζεται τα πάντα: από τα επείγοντα και τις νοσηλείες, μέχρι τις βάρδιες του προσωπικού.

Για να τα καταφέρουμε, αξιοποιήσαμε το Claude της Anthropic ως ψηφιακό βοηθό. Το χρησιμοποιήσαμε στο debugging και μας καθοδήγησε στο να στήσουμε σωστά το Docker.